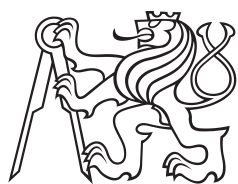


Bachelor Thesis



**Czech
Technical
University
in Prague**

F3

**Faculty of Electrical Engineering
Department of Radioelectronics**

CAN Bus Data Analysis System for Automotive Monitoring

Jakub Klášterka

Supervisor: Ing. Ondřej Nentvich, Ph.D.

Field of study: Electronics and Communication

May 2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Klásterka** Jméno: **Jakub** Osobní číslo: **525812**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra radioelektroniky**
Studijní program: **Elektronika a komunikace**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Systém analýzy dat ze sběrnice CAN pro monitorování automobilů

Název bakalářské práce anglicky:

CAN Bus Data Analysis System for Automotive Monitoring

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Ondřej Nentvich, Ph.D. katedra radioelektroniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **04.02.2026**

Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

_____ Datum převzetí zadání

Klásterka Jakub

_____ Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Klásterka** Jméno: **Jakub** Osobní číslo: **525812**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra radioelektroniky**
Studijní program: **Elektronika a komunikace**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Systém analýzy dat ze sběrnice CAN pro monitorování automobilů

Název bakalářské práce anglicky:

CAN Bus Data Analysis System for Automotive Monitoring

Pokyny pro vypracování:

- 1) Perform research on commonly used communication buses in vehicles, focusing on the CAN bus and the OBD-II diagnostic standard. The research should cover the following areas:
 - Overview of commonly used buses (e.g., CAN, LIN, FlexRay, MOST, Automotive Ethernet)
 - OBD-II diagnostic standard and related protocols (especially ISO 15765 and SAE J1979)
 - Description of the CAN 2.0A/B protocol
 - Diagnostic services and vehicle operating values
- 2) Review existing devices for analyzing and recording data from the CAN bus.
- 3) Design a device capable of recording data from the CAN bus via the OBD-II diagnostic connector and storing this data in local or cloud storage.
- 4) Design a method for visualizing the acquired data using a graphical display or web interface, implemented either directly on the microcontroller or via remote storage. The visualization should include key vehicle diagnostic and operating data.
- 5) Analyze the security risks of the designed device and propose appropriate measures to ensure data protection and bus integrity.

Seznam doporučené literatury:

- [1] PARET, Dominique. Multiplexed Networks for Embedded Systems: CAN, LIN, FlexRay, Safe-by-Wire.. John Wiley, 2007. ISBN 978-0-470-51170-1.
- [2] KLEIDERMACHER, David a KLEIDERMACHER, Mike. Embedded systems security: practical methods for safe and secure software and systems development. Amsterdam: Elsevier, 2012. ISBN 978-0-12-386886-2.
- [3] ZÁHLAVA, Vít. Návrh a konstrukce desek plošných spojů: principy a pravidla praktického návrhu. Praha: BEN - technická literatura, 2010. ISBN 978-80-7300-266-4.
- [4] Mastering MQTT – Your Ultimate Tutorial for MQTT [online]. Available from: <https://assets.emqx.com/resources/ebooks/Mastering%20MQTT-Your%20Ultimate%20Tutorial%20for%20MQTT.pdf>

DECLARATION

I, the undersigned

Student's surname, given name(s): Klášterka Jakub
Personal number: 525812
Programme name: Electronics and Communications

hereby declare that the Bachelor's Thesis titled

CAN Bus Data Analysis System for Automotive Monitoring

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 03.05.2026

Jakub Klášterka

.....
student's signature

Acknowledgements

I would like to thank my supervisor, Ing. Ondřej Nentvich, Ph.D., for his invaluable guidance and continuous support throughout this thesis, often helping me outside of his regular working hours, including weekends. I am grateful for his empathy, knowledge, and eagerness to help with all the difficulties encountered during the work.

I would also like to thank my external supervisor, Ing. Ludmila Musalová Stoklasová, from the Technical Development Department at Škoda Auto, for her guidance and consultations.

Finally, I am grateful to my colleagues for their assistance and valuable advice while working on this Bachelor thesis.

Poděkování

Chtěl bych velice poděkovat svému vedoucímu, Ing. Ondřeji Nentvichovi, Ph.D., za jeho odborné vedení a neustálou podporu v průběhu celé práce, i nad rámec jeho pracovních hodin a během víkendů. Velmi si vážím jeho vstřícnosti, odborných znalostí a ochoty pomoci při řešení všech obtíží, s nimiž jsem se během její tvorby setkal.

Dále také děkuji své externí konzultantce, Ing. Ludmile Musalové Stoklasové z oddělení technického vývoje společnosti Škoda Auto, za její vedení a konzultace.

Na závěr děkuji svým kolegům za pomoc a cenné rady při tvorbě této bakalářské práce.

Abstract

The Bachelor thesis focuses on developing a prototype of a diagnostic device for vehicle telematics which can monitor vehicle data in real time wirelessly on a web dashboard. The prototype device contains a custom-designed PCB with an ESP32-S3 microcontroller, a CAN FD controller, a SIMCom external LTE & GPS module, and a TPS54331 power supply.

The theoretical part contains a discussion of the OBD-II standards and specifications, and the CAN bus protocol architecture. A comparison of several different automotive communication buses, including FlexRay, LIN, and Automotive Ethernet, is provided as a foundation for understanding the automotive environment. There is also a marketing overview of existing diagnostic devices.

The practical part of the thesis demonstrates the custom PCB design of the device, firmware development, server setup, and the whole setup of the device, including LTE initialization, reading CAN messages, and providing steps for security measures and power management. It also describes the developed dashboard for the visualization of vehicle data and the validation of the acquired data.

The thesis also offers an idea for future implementation of a diagnostic interface for flashing vehicle electronic control units.

Keywords: ESP32, CAN bus, OBD-II, automotive diagnostics, microcontroller, PCB design, web server, wireless technology

Supervisor:

Ing. Ondřej Nentvich, Ph.D.
Department of Radioelectronics,
Faculty of Electrical Engineering,
CTU Prague

Abstrakt

Bakalářská práce se zaměřuje na vývoj prototypu diagnostického zařízení, které dokáže bezdrátově sledovat telematická data z vozidla a vizualizovat je pomocí webového rozhraní. Prototyp zahrnuje vlastní vývoj desky plošných spojů s mikrokontrolérem ESP32-S3, CAN FD převodníkem, externím LTE a GPS modulem SIMCom a napájecím zdrojem TPS54331.

Teoretická část se věnuje popisu standardů a specifikací OBD-II a architektuře protokolu CAN bus. Obsahuje také srovnání několika různých komunikačních sběrnic ve vozidle, včetně FlexRay, LIN a Automotive Ethernet, jako základ pro pochopení průmyslových standardů v automotive prostředí. Tato část přináší také tržní přehled existujících diagnostických zařízení.

Praktická část popisuje návrh vlastní desky plošných spojů (DPS), vývoj firmwaru, přípravu serveru a celkovou konfiguraci zařízení. Ta zahrnuje inicializaci LTE komunikace, vyčítání zpráv ze sběrnice CAN, řešení softwarového zabezpečení a řízení spotřeby energie. Dále je zde popsán vývoj webového rozhraní pro vizualizaci naměřených dat a jejich validace.

Práce také obsahuje možnost budoucího rozšíření systému, spočívající v implementaci diagnostického rozhraní pro kódování elektronických řídicích jednotek.

Klíčová slova: ESP32, sběrnice CAN, OBD-II, automobilová diagnostika, mikrokontrolér, návrh DPS, webový server, bezdrátové technologie

Překlad názvu: Systém analýzy dat ze sběrnice CAN pro monitorování automobilů

Contents

1 Introduction	1		
1.1 Motivation	1		
1.2 Objectives	1		
2 Theoretical Background	3		
2.1 Controller Area Network	3		
2.1.1 History and Development	3		
2.1.2 Network Architecture	4		
2.1.3 CAN Message Frame Structure	7		
2.1.4 Reading and Decoding CAN Messages	11		
2.1.5 CAN Protocol Variants	12		
2.1.6 Electronic Control Unit	13		
2.1.7 Typical ECUs in Vehicles	14		
2.1.8 Vehicle Network Architecture	14		
2.1.9 Other Serial Buses in Vehicles	15		
2.2 OBD-II Standard	18		
2.2.1 Overview and Standardization	18		
2.2.2 OBD II Physical Connector	19		
2.2.3 Parameter IDs	20		
2.3 Market Analysis of Existing OBD-II Solutions	22		
3 Design and Implementation of the Device	25		
3.1 Introduction	25		
3.2 Hardware Design	25		
3.2.1 System Requirements and Component Selection	25		
3.2.2 Circuit Design	28		
3.2.3 PCB Design	31		
3.2.4 System Pin Assignment	33		
3.3 Firmware Development	34		
3.3.1 Software Architecture	34		
3.3.2 System Flow Diagram	34		
3.3.3 CAN Controller Initialization	37		
3.3.4 OBD-II PID Decoding	38		
3.3.5 LTE Communication	39		
3.3.6 GPS Acquisition	40		
3.4 Cloud Infrastructure	41		
3.4.1 Deployment Platform	41		
3.4.2 REST API Design	41		
3.4.3 Data Storage	42		
3.4.4 Repository	42		
3.5 Security Analysis	43		
3.5.1 LTE Channel Security	43		
3.5.2 Local Wi-Fi Channel Security	45		
3.6 Web Dashboard	46		
3.6.1 Dashboard Overview	46		
3.6.2 Advanced Diagnostics Interface	48		
3.7 Testing and Validation	49		
3.7.1 End-to-End Transmission Test	49		
3.7.2 Vehicle CAN Communication	51		
3.7.3 Power Measurements	54		
3.8 Final Devices	56		
3.8.1 Web Server Endpoint	56		
3.8.2 Power Management	57		
3.8.3 Data Payload	57		
4 Conclusion	59		
Bibliography	61		
A Component Datasheets	65		
B Complete Hardware Schematic	67		
C Dashboard Screens	71		
Cockpit Screen	71		
Vehicle Overview Screen	73		
Live Graphs Screen	74		
Fuel Consumption Screen	75		
GPS Map Screen	76		
Alarms Screen	77		
Advanced Diagnostics Interface	78		
D Test Drive Cockpit Photographs	83		
45 km h ⁻¹ Cruise	83		
Full-Throttle Acceleration to 100 km h ⁻¹	84		
50 km h ⁻¹ Steady-State	84		
High-RPM Stationary Run	84		
E Repository Structure	85		
F Usage of Artificial Intelligence tools	87		

Figures

2.1 CAN protocol architecture [20] ..	4	C.1 Cockpit screen — live radial gauges and summary widgets	72
2.2 CAN bus topology architecture [4]	5	C.2 Vehicle Overview screen — PID values and CSV export	73
2.3 CAN bus differential voltage levels [5]	6	C.3 Live Graphs screen — time series plots with zoom support	74
2.4 CAN bus data frame [39]	8	C.4 Fuel Consumption screen — instantaneous and average counters	75
2.5 Cyclic Redundancy Check diagram [21]	9	C.5 GPS Map screen — real-time vehicle position and route export .	76
2.6 Engine control unit [25]	13	C.6 Alarms screen — configurable threshold limits	77
2.7 ECUs topology in a car [27]	13	C.7 Login screen — access restriction for the diagnostics interface	78
2.8 ECU topology in vehicle [26] (bus layout is for illustration only)	14	C.8 Diagnostic interface	79
2.9 OBD-II most common locations [37]	19	C.9 Serial Monitor	80
2.10 OBD-II Connector pinout [9] ..	19	C.10 UDS & Coding — ISO-TP request builder	80
2.11 OBDeleven diagnostic dongle [22]	22	C.11 Live monitor — real-time CAN traffic display	81
2.12 Generic ELM327 Bluetooth adapter [17]	23	D.1 Instrument cluster during 45 km h ⁻¹ cruise	83
2.13 SIZZAPP 4G GPS OBD tracker [30]	23	D.2 Instrument cluster during full-throttle acceleration to 100 km h ⁻¹	84
2.14 T-Mobile Smart Car OBD-II device [15]	24	D.3 Instrument cluster during 50 km h ⁻¹ steady-state cruise	84
3.1 CAN Bus Sniffer block diagram.	28	D.4 Instrument cluster during stationary high-RPM engine run ..	84
3.2 PCB layout of 4-layer board ...	32		
3.3 PCB layout showcasing the short high-frequency current loop path .	32		
3.4 Main loop flow diagram	36		
3.5 Illustration photos of LTE antenna (left)[34] and GNSS antenna (right)[36]	40		
3.6 Main Cockpit screen featuring live radial gauges and summary widgets	47		
3.7 Recorded engine speed and vehicle speed over the test session	52		
3.8 Throttle position, engine load, and intake manifold pressure.	53		
3.9 Coolant temperature, intake air temperature, ambient temperature, and mass air flow rate.	53		
3.10 Current draw during LTE module initialization and TCP payload transmission.	54		
3.11 System power consumption comparison between light sleep and deep sleep modes.	55		

Tables

2.1 Comparison of CAN Protocol Variants	12
2.2 Common OBD-II Standard PIDs	20
3.1 Key features of the ESP32-S3-MINI-1 module (N8 variant)	26
3.2 ESP32-S3 GPIO Pin Assignment by Functional Block	33
3.3 REST API Endpoints	41
3.4 Dashboard Screen Overview	46
3.5 OBD-II parameters acquired during the test drive	51
3.6 Measured current draw per operating phase.	57
3.7 Projected cellular data consumption (4 h/day, $n = 9$ PIDs).	58
E.1 Repository Structure	85

List of Abbreviations

Abbreviation	Description
ADAS	Advanced Driver Assistance Systems
ADC	Analog-to-Digital Converter
AP	Access Point
API	Application Programming Interface
AT	Attention (standard modem command prefix)
BOM	Bill of Materials
BCM	Body Control Module
CARB	California Air Resources Board
CGW	Central Gateway
CPU	Central Processing Unit
DNS	Domain Name System
DNP	Do Not Populate
DTC	Diagnostic Trouble Code
EMI	Electromagnetic Interference
EOBD	European On-Board Diagnostics
ESD	Electrostatic Discharge
FIFO	First-In, First-Out
FSM	Finite State Machine
GPIO	General-Purpose Input/Output
GPRS	General Packet Radio Service
GPS	Global Positioning System

Abbreviation	Description
GPX	GPS Exchange Format
GSM	Global System for Mobile Communications
HVAC	Heating, Ventilation, and Air Conditioning
I²C	Inter-Integrated Circuit
IDE	Identifier Extension
ISO	International Organization for Standardization
ISO-TP	ISO Transport Protocol (ISO 15765-2)
ISR	Interrupt Service Routine
JSON	JavaScript Object Notation
LDO	Low-Dropout Regulator
LiDAR	Light Detection and Ranging
LTE	Long Term Evolution
MAC	Medium Access Control
MAF	Mass Air Flow
MAP	Manifold Absolute Pressure
OSI	Open Systems Interconnection
PCB	Printed Circuit Board
PID	Parameter Identifier
PWM	Pulse Width Modulation
QFN	Quad Flat No-leads
RAM	Random Access Memory
REST	Representational State Transfer
RMT	Remote Control Peripheral
ROM	Read-Only Memory
RTR	Remote Transmission Request
SAE	Society of Automotive Engineers
SIM	Subscriber Identity Module
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
SSID	Service Set Identifier

Abbreviation	Description
TDM	Time-Division Multiplexing
TLS	Transport Layer Security
TWAI	Two-Wire Automotive Interface
UART	Universal Asynchronous Receiver-Transmitter
UDS	Unified Diagnostic Services
UTP	Unshielded Twisted Pair
VAG	Volkswagen Aktiengesellschaft
WPA2-PSK	Wi-Fi Protected Access 2 — Pre-Shared Key

Chapter 1

Introduction

In the modern automotive industry, the usage of electronic control units (ECUs) and standardized communication protocols is essential for diagnostics and monitoring.

The On-Board Diagnostics II (OBD-II) standard, which has been required in all vehicles since 1996, provides a standard interface that allows access to vehicle data and diagnostic information through the use of the Controller Area Network (CAN) bus protocol. This bachelor thesis details the design of a diagnostic tool utilizing an ESP32 microcontroller to communicate with the OBD-II interface of vehicles utilizing the CAN bus protocol.

This diagnostic prototype allows real-time tracking of vehicle diagnostics and provides visualized data via a web-based dashboard.

1.1 Motivation

The motivation for creating this thesis is driven by several factors:

- Cost-effective diagnostics solutions for automotive monitoring.
- Learning and education — the need for modern electrical engineering students to understand automotive communication protocols.
- Fleet management applications — providing real-time monitoring of vehicle diagnostics, allowing better fleet maintenance and efficiency.

1.2 Objectives

- Research the OBD-II standard and CAN bus protocol architecture.
- Design the hardware interface to connect the ESP32 to the CAN bus communication.
- Implement software to read and convert raw data to specific vehicle parameters.
- Develop a web-based dashboard to present remote data.

Chapter 2

Theoretical Background

2.1 Controller Area Network

The Controller Area Network (CAN) is a serial communication bus designed for robust message exchange between Electronic Control Units (ECUs). It was initially engineered for the automotive environment, where the protocol uses a multi-master architecture in which devices broadcast messages to the entire network without a central host computer. This decentralized approach serves as the vehicle's nervous system, enabling components such as engine management, transmission control, and safety systems to share sensor data in real time. While developed for combustion-engine vehicles, the protocol serves as the networking standard for electric vehicles, marine electronics, and industrial automation [6].

2.1.1 History and Development

In the early 1980s, the automotive industry faced a critical engineering challenge known as the "wiring harness crisis". As manufacturers integrated electronic features such as Anti-lock Brake Systems (ABS), electronic fuel injection, and power windows, the reliance on point-to-point analog wiring became unsustainable. By 1983, the wiring harness in luxury vehicles weighed over 50 kg and spanned kilometers, making it the third-heaviest and most expensive component in the assembly [3]. This complexity introduced manufacturing difficulties and reliability issues caused by connector corrosion.

An innovative solution was introduced by Robert Bosch GmbH,¹ initiating the CAN protocol in 1983 [3]. The solution consisted of replacing individual wiring with a single twisted-pair bus system, thereby reducing weight and complexity while maintaining high noise immunity. Intel produced the first CAN controller chip, the 82526, in 1987, followed by the Philips 82C20 [2]. The technology entered mass production in 1991 with the Mercedes-Benz W140 S-Class, which utilized the bus to interconnect the engine management and transmission control units.

¹*Gesellschaft mit beschränkter Haftung*, the German form of a limited liability company.

2.1.2 Network Architecture

The CAN bus is a multi-master serial line that uses differential signaling to determine the logical level of the transmitting signal. The architecture refers to the ISO/OSI Reference Model, specifically implementing the Physical Layer and Data Link Layer. The transmission lines are designated as CAN_L (CAN Low) and CAN_H (CAN High).

The protocol is internationally standardized under the ISO 11898 series:

- **Data Link Layer (ISO 11898-1):** Governs the logical handling of messages, including frame construction, error detection, and the arbitration logic that manages bus access.
- **Physical Layer (ISO 11898-2):** Specifies the electrical characteristics of the signals and the physical medium, which consists of two transmission lines designated as High and Low [6].

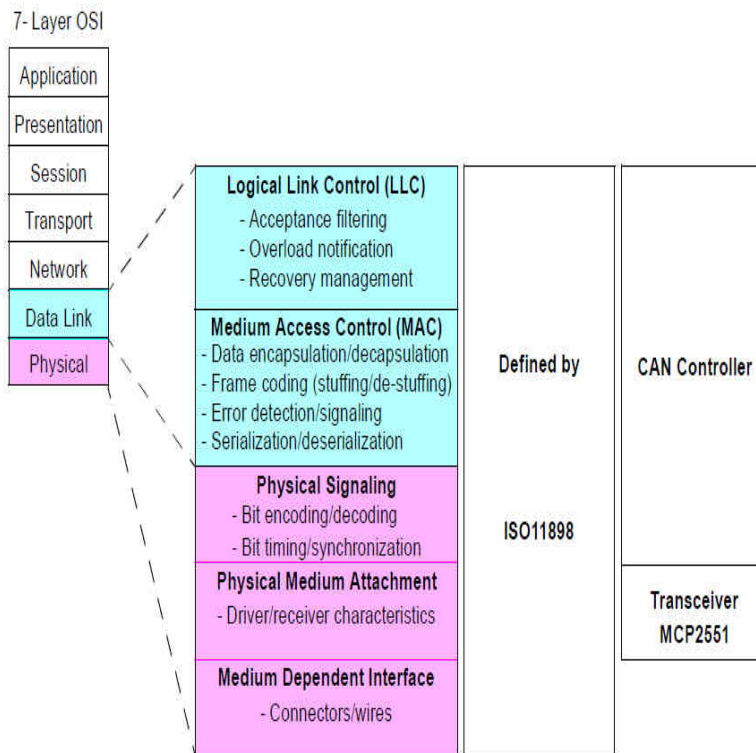


Figure 2.1: CAN protocol architecture [20]

■ Bus Topology

As mentioned before, the CAN bus is a multi-master line. It is designed in a bus topology, which has some key advantages over widely used star topology solutions. The star topology is often used for single-master applications, such as SPI or in many network topology solutions. In contrast, as a multi-master bus, CAN uses a 2-wire bus topology where every component is attached and each node can serve as master or slave, depending on whether it is requesting or sending data. Bus topology provides significant wiring savings. Each node consists of a Microcontroller, a CAN Controller, and a CAN Transceiver:

- **Microcontroller (MCU):** The "brain" of the node that interprets received data and decides what information to transmit (Speed, RPM, etc.).
- **CAN Controller:** Often integrated into the MCU, managing bit representation, error handling, and message buffering according to ISO 11898-1.
- **CAN Transceiver:** Acts as the interface between the controller and physical bus. It converts digital logic into differential electrical signals [35].

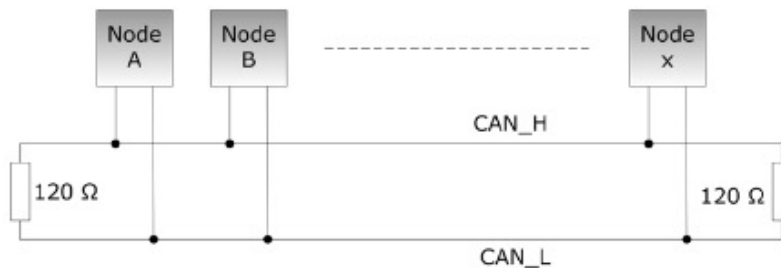


Figure 2.2: CAN bus topology architecture [4]

Physical Layer

The Physical Layer is responsible for the actual electrical transmission of data over the CAN bus medium. It defines the voltage levels, signal timing, and physical characteristics of the bus.

The key advantage of the differential bus is its immunity to Electromagnetic Interference (EMI). When a receiving node determines the voltage level on the line, it utilizes the differential voltage V_{diff} between V_{CAN_H} and V_{CAN_L} , as shown in Equation 2.1:

$$V_{diff} = V_{CAN_H} - V_{CAN_L} \quad (2.1)$$

This mechanism ensures that potential signal distortion affects both lines equally, effectively canceling out the interference. The differential voltage levels are illustrated in Figure 2.3. As a serial data bus, correct impedance matching is critical to avoid signal reflections. The characteristic impedance of a standard CAN bus cable is $120\ \Omega$; termination resistors at both ends of the bus must therefore match this value. Placing two $120\ \Omega$ termination resistors, denoted as R_1 and R_2 , at the ends of the bus, results in a parallel equivalent resistance R_{term} of $60\ \Omega$, as derived by Equation 2.2:

$$R_{term} = \frac{R_1 \cdot R_2}{R_1 + R_2} = \frac{120 \cdot 120}{120 + 120} = 60\ \Omega \quad (2.2)$$

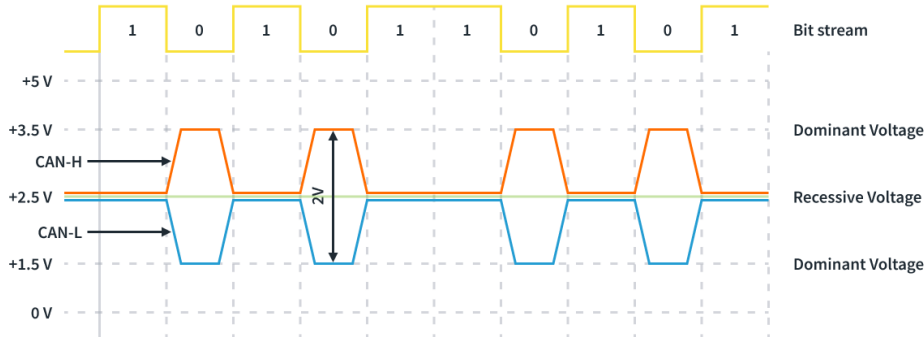


Figure 2.3: CAN bus differential voltage levels [5]

■ Data Link Layer

The Data Link Layer, defined by ISO 11898-1, manages frame construction, bus access arbitration, and error detection. It is functionally divided into two sub-layers: Logical Link Control (LLC) and Medium Access Control (MAC).

- **Logical Link Control:** The LLC sub-layer manages message filtering and acceptance. It determines which received messages are relevant to the host MCU based on the message Identifier (ID). Additionally, it handles overload notifications [7].
- **Medium Access Control:** The MAC sub-layer controls the core protocol rules:
- **Arbitration:** Manages non-destructive bitwise arbitration using CSMA/CD+AMP.
- **Frame Coding:** Performs bit stuffing, inserting a complementary bit after every five consecutive identical bits [18].
- **Error Detection:** Implements error checking through Cyclic Redundancy Check (CRC) calculation.

■ 2.1.3 CAN Message Frame Structure

■ Bus States and Bit Levels

To understand the operation of CAN frames, it is essential to examine how bits are represented on the bus. The CAN protocol utilizes two distinct states:

Dominant bit (logic 0): CAN_H goes to about 3.5 V while CAN_L drops to 1.5 V, making a differential voltage of roughly 2 V. It is called "dominant" because it electrically overpowers the recessive state.

Recessive bit (logic 1): Both lines stay at around 2.5 V, so the differential voltage is nearly 0 V. This is "recessive" because a dominant bit from any node will override it [18].

When multiple nodes try to transmit at once, this dominant/recessive setup is what makes arbitration work – dominant bits always override recessive bits on the shared line.

■ Message-Based Communication

The CAN protocol is based on message-framed communication. Each frame has an ID that specifies what the message consist of, not who sent it or where it is going. Nodes filter messages by content – they only pay attention to what matters to them. ISO 11898-1 defines four frame types: Data Frames, Remote Frames, Error Frames, and Overload Frames. Two formats are used: Standard Frame (CAN 2.0A) with an 11-bit ID, or Extended Frame (CAN 2.0B) with a 29-bit ID for more complex protocols like SAE J1939 [6].

Standard Data Frame Fields

According to [24], a standard CAN Data Frame has seven fields, illustrated in the waveform in Figure 2.4.

1. **Start of Frame (SOF):** A single dominant bit (0) that marks the start of transmission and synchronizes all nodes.
2. **Arbitration Field:** Contains the 11-bit Identifier and RTR bit. The ID determines priority – lower values mean higher priority due to the wired-AND logic. The RTR (Remote transmission request) bit is dominant (0) for Data Frames and recessive (1) for Remote Frames.
3. **Control Field:** Includes the IDE bit (dominant for Standard Frames), a reserved bit (r0), and the 4-bit Data Length Code (DLC) that specifies payload size (0–8 bytes).
4. **Data Field:** The actual payload, transmitted MSB first, from 0 to 8 bytes.
5. **CRC Field:** A 15-bit Cyclic Redundancy Check followed by a recessive delimiter.
6. **ACK Field:** Contains the ACK Slot and delimiter. The transmitter sends a recessive bit, which gets overwritten by a dominant bit if any node successfully receives the frame [24].
7. **End of Frame (EOF):** Seven consecutive recessive bits marking the end of transmission.

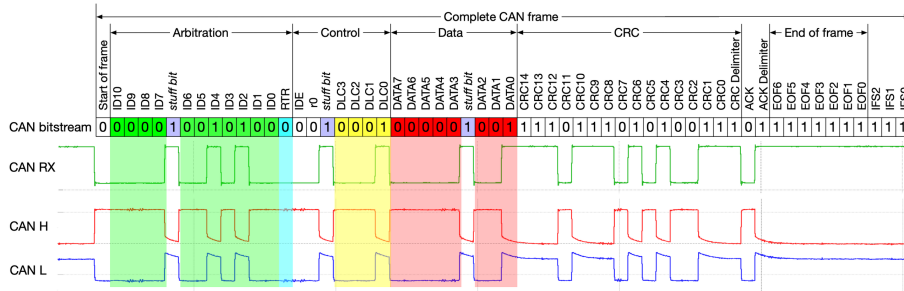


Figure 2.4: CAN bus data frame [39]

Cyclic Redundancy Check

The CRC detects transmission errors using polynomial math. The transmitter calculates a 15-bit checksum with the generator polynomial from ISO 11898-1:

$$G(x) = x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1 \quad (2.3)$$

Mathematical principle

CAN represents data bits as polynomials - each bit position becomes a term. For example, if you have the bit sequence 10110, it translates to:

$$M(x) = x^4 + x^2 + x \quad (2.4)$$

The CRC calculation has three steps:

1. Take the message polynomial and multiply it by x^{15} (basically shifting it left by 15 bits)
2. Divide this by the generator polynomial using modulo-2 math:

$$\frac{x^{15} \cdot M(x)}{G(x)} = Q(x) + \frac{R(x)}{G(x)} \quad (2.5)$$

where $Q(x)$ is what you get from the division and $R(x)$ is what's left over

3. That leftover part $R(x)$ becomes your 15-bit CRC [24].

Modulo-2 arithmetic just means using XOR operations - so $1 + 1 = 0$ and $1 + 0 = 1$, with no carrying between bits. When a node receives the frame, it does the same division on all the data including the CRC. If everything transmitted correctly, the remainder comes out to zero. Any other result means there's an error somewhere.

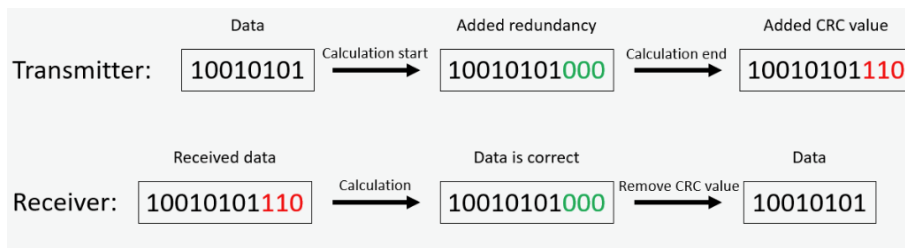


Figure 2.5: Cyclic Redundancy Check diagram [21]

Extended Data Frame Differences

The Extended Frame (CAN 2.0B) mainly differs in the Arbitration Field. It uses a 29-bit Identifier split into an 11-bit Base ID and 18-bit Extended ID. The IDE bit is recessive (1) to distinguish it from Standard Frames, and an SRR (Substitute Remote Request) bit replaces the RTR bit in the first part of the field.

CAN uses bit stuffing to keep nodes synchronized. From the SOF through the CRC, whenever five consecutive identical bits appear, the transmitter automatically inserts a complementary bit (stuff bit). This ensures regular signal transitions for clock recovery and prevents data from being confused with error frames or EOF sequences.

CAN uses CSMA/CD+AMP (Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority) to handle bus access. When multiple nodes try to transmit at the same time, the system resolves the conflict without losing data - the highest priority message keeps transmitting while others back off.

Bitwise Arbitration

The arbitration works because of the bus's wired-AND logic: a Dominant bit (0) always overwrites a Recessive bit (1). When the bus is idle, several nodes can start transmitting their Start of Frame simultaneously. As they send their Identifier bits one by one, each node monitors the bus to check if what it's sending matches what's actually on the line.

If a node sends a Recessive bit (1) but sees a Dominant bit (0) on the bus, it knows another node is sending a higher priority message. That node immediately stops transmitting and switches to receive mode to listen to the winning message. This happens in real-time during the arbitration field.

Message Priority

Since Dominant bits (0) overwrite Recessive bits (1), messages with lower binary IDs have higher priority. For example, a node sending ID 0x100 will win against one sending 0x110 because at the first differing bit position, the 0 dominates the 1. This ensures safety-critical messages (like engine timing or ABS data) always get through before less important ones (like infotainment) [14, 10].

2.1.4 Reading and Decoding CAN Messages

CAN is message-based, not address-based like TCP/IP. Frames are not sent to specific nodes - instead, the Identifier defines what the message contains, and it's broadcast to everyone on the network [35].

Hardware Filtering

Every node physically receives every message on the bus, so there needs to be a way to filter out irrelevant data. The CAN controller handles this using Acceptance Filters and Masks. It compares incoming message IDs against configured filters - if there's a match, the message goes into a buffer and triggers an interrupt for the microcontroller. Non-matching messages get discarded at the hardware level, which saves processing power.

Decoding Raw Data

The data field in a CAN frame contains up to 8 bytes of raw hexadecimal data that isn't human-readable. To make sense of it, you need to decode it into actual engineering values (RPM, temperature in °C, speed in km h⁻¹, etc.). This is usually done with a CAN Database (DBC) file, which provides the decoding rules.

To extract a specific signal from the raw payload, you need:

1. **Bit Start and Length:** Where the signal sits within the 64-bit data field
2. **Byte Order:** Whether it's Intel (Little-Endian) or Motorola (Big-Endian) format
3. **Scaling and Offset:** Convert the raw value to physical units using:

$$\text{Physical Value} = (\text{Raw Value} \times \text{Scale}) + \text{Offset} \quad (2.6)$$

For example, if the raw 8-bit value for Engine Speed is 0x32 (decimal 50), with a scaling factor of 1 and offset of 0, the decoded value is 50 RPM [7].

2.1.5 CAN Protocol Variants

The CAN protocol exists in several variants, each developed for specific application requirements — from fault-tolerant body electronics to high-speed powertrain control. Beyond the automotive industry, CAN bus is widely used in robotics, marine electronics, and industrial automation.

- **Low-Speed CAN (ISO 11898-3)** — used in body electronics where reliability matters more than speed. If one of the bus wires gets cut or shorts to ground, the transceivers automatically switch to single-wire mode. Operates between 40 kbit/s and 125 kbit/s.
- **High-Speed CAN (ISO 11898-2)** — the most common variant, used in powertrain and chassis systems and forming the physical layer for OBD-II, SAE J1939, and CANopen. Runs up to 1 Mbit/s [7].
- **CAN FD (ISO 11898-1:2015)** — developed for EVs and ADAS requiring higher bandwidth with up to 8 Mbit/s. It can be increased from 8 bytes to during the data phase and payload 64 bytes [12].
- **CAN XL (ISO 11898-1:2024)** — bridges CAN FD and Automotive Ethernet in zonal architectures. Offers speeds of 10–20 Mbit/s and payload up to 2048 bytes, supporting TCP/IP encapsulation directly in CAN XL frames [41].

A comparison of all variants is provided in Table 2.1.

Table 2.1: Comparison of CAN Protocol Variants

Feature	Low-Speed CAN	High-Speed CAN	CAN FD	CAN XL
Standard (ISO)	11898-3	11898-2	11898-1:2015	11898-1:2024
Speed	125 kbit/s	Up to 1 Mbit/s	5–8 Mbit/s	10–20 Mbit/s
Payload	8 bytes	8 bytes	64 bytes	2048 bytes
Usage	■ BCM ■ Lighting ■ HVAC	■ Powertrain ■ ABS, ECU ■ OBD-II	■ EVs, ADAS ■ ECU flashing	■ Zonal arch. ■ Ethernet bridge
Key Features	■ Fault-tolerant	■ Low cost ■ Robust	■ High payload ■ High speed	■ Highest payload ■ Highest speed

2.1.6 Electronic Control Unit

An Electronic Control Unit (ECU) is the brain of a vehicle - it reads sensor data and tells components what to do. Modern cars can have up to 120 ECUs, each controlling something specific like the engine, transmission, brakes, or climate control. Inside each ECU is a microcontroller running software that processes sensor data and sends commands to actuators that actually make things happen [13].

ECUs communicate through serial buses, with CAN bus being the most common. This allows different systems to share information without complex wiring. The Central Gateway (diagnostic code 0x0019) acts as the main hub, routing and filtering data between different bus networks and connecting to the OBD-II port for external diagnostic access.



Figure 2.6: Engine control unit [25]

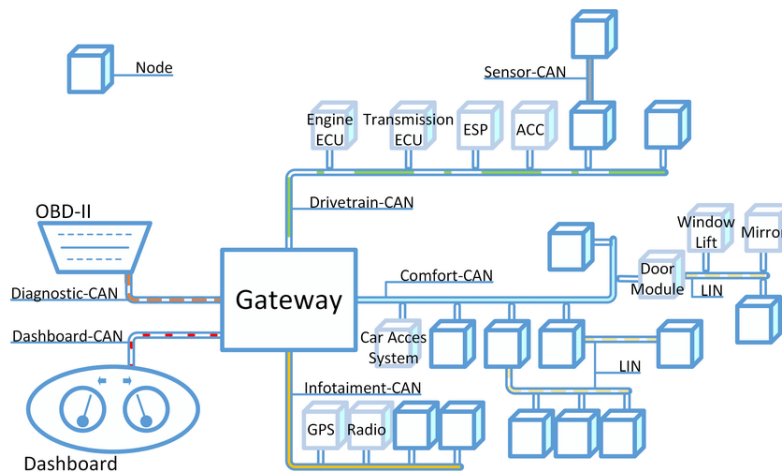


Figure 2.7: ECUs topology in a car [27]

2.1.7 Typical ECUs in Vehicles

- **ECM - Engine Control Module** — main powertrain controller handling fuel injection and ignition timing over High-Speed CAN [7]
- **TCU - Transmission Control Unit** — manages transmissions by monitoring vehicle speed and engine load, coordinating with the ECM over the powertrain CAN bus
- **BCM - Body Control Module** — controls comfort features via Low-Speed CAN (ISO 11898-3) or LIN bus [7]
- **OCU - Onboard Connectivity Unit** — handles GPS, cellular data, and Wi-Fi, tied to the Central Gateway to prevent unauthorised access
- **BMS - Battery Management System** — monitors battery state of charge, health, and cell temperatures in EVs, often using CAN FD
- **CGW - Central Gateway** — routes traffic between bus networks and serves as the only externally accessible module via the OBD-II port [7]

2.1.8 Vehicle Network Architecture

The vehicle network uses a hierarchical topology. At the top, the **Central Gateway** connects all vehicle networks, each dedicated to a specific domain:

- **Powertrain CAN - Green** — engine control, transmission, fuel system
- **Chassis CAN - Red** — ABS, ESP, steering, suspension
- **Body/Comfort CAN - Dark blue** — door modules, climate, lighting
- **Infotainment - Light blue** — multimedia, navigation, connectivity

Domain controllers communicate with the Gateway over High-Speed CAN, while slave modules use lower-cost protocols, such as LIN bus [7].

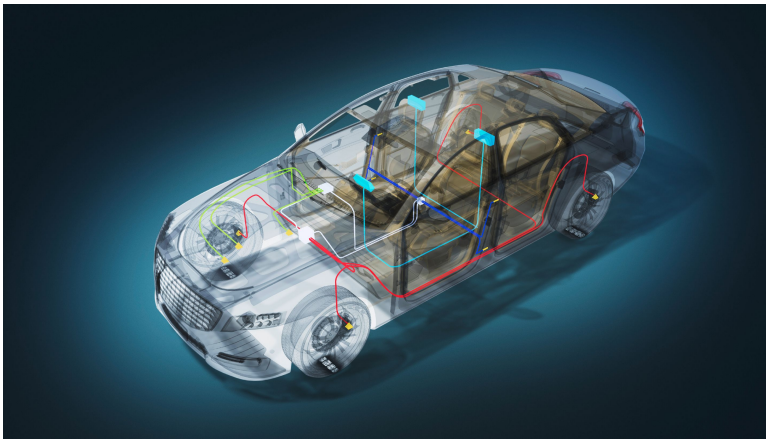


Figure 2.8: ECU topology in vehicle [26] (bus layout is for illustration only)

■ 2.1.9 Other Serial Buses in Vehicles

The automotive industry has an array of serial busses beyond just CAN. However, there exist multiple other common forms of automotive communications found throughout the entire vehicle architecture.

■ Local Interconnect Network

LIN is a single wire interface wired at 12 volts with ground; whereas CAN operates at mean differential using two wire pairs. It is designed as a master/slave architecture where one master (generally tied into the main CAN network) controls communication with up to 16 slave devices.

Speed: LIN supports data rates from 1 Kbps to 20 Kbps, using a time scheduled message instead of a competition based arbitration system - the master performs all of the scheduling for data sending times for all devices on a LIN Network.

Usage: LIN is best used in Body Control applications such as power windows, outside mirrors, locks and HVAC actuators. Generally speaking, the functions provided by LIN devices consist mainly of low-speed, non-safety-critical mechanical actuators. LIN provides significant cost savings due to its simplified wiring and the reduced number of components required, with no licensing fee associated with any LIN applications [7].

■ Automotive Ethernet

Today's vehicles encounter limitations in bandwidth, as the sheer volume of data created by advanced cameras, LiDAR sensors and high-resolution displays has reached an overwhelming level for traditional buses. From this reason, automotive manufacturers have begun deploying Ethernet as the communication medium for these mission-critical applications. Automotive Ethernet provides an adaptation of existing IP networking technologies specifically tailored for use in vehicles. The Automotive Ethernet is built off of standard networking technology modified for the conditions found in a vehicle including twisted pair cables that were adapted to support the extreme temperatures of a vehicle and the electrical noise inside the vehicle. Unlike the shared bus of CAN, Ethernet is based on a point-to-point connection or switched network topology.

Speed: Ethernet supports much higher speeds than the 1Mbit/s or lower that CAN provides. It can support speeds from 100Mbit/s to greater than 1Gbit/s and can transmit multiple video streams and data.

Usage: This protocol is primarily utilized by Advanced Driver Assistance Systems (ADAS) and infotainment systems requiring continuous high-bandwidth data transmission. Although Ethernet has much higher throughput capabilities than CAN, Ethernet does not have the built-in safety mechanisms or timing predictability found in CAN. Vehicles use both protocols - CAN handles timing-critical control and Ethernet manages high volume data.

Media Oriented Systems Transport

Originally designed for vehicle entertainment and multimedia systems, MOST is able to transfer data types that would normally cause overloads to traditional automotive buses. This bus adopts a ring topology to connect devices compared with the CAN straight-line bus architecture found on most vehicles. MOST uses a ring topography for devices connected in a closed loop, unlike CAN which uses a linear bus architecture. The initial implementations used plastic optical fibre (POF) cable to deliver good immunity to electromagnetic interference (EMI), but later generations introduced unshielded twisted pair (UTP) copper cabling to reduce costs at the expense of electric noise immunity.

Speed: MOST has been through three distinct generations: MOST25 (25Mbit/s), MOST50 (50Mbit/s) and MOST150 (150Mbit/s). All generations use time-division multiplexing (TDM) to allocate bandwidth, and ensure that there are no interruptions in the streaming of audio and video products that are dependent on users having a continuous, jitter free playback experience [1].

Usage: The three premier automotive manufacturers, Mercedes-Benz, Audi and BMW have heavily implemented MOST in high-end audio systems, navigation systems, and rear entertainment systems, respective applications. But MOST is now in the process of being phased out, for most of the industry, as most manufacturers have begun to transition to Automotive Ethernet in order to support both more flexible architecture as defined by modern software defined vehicles (SDV).

FlexRay

FlexRay was created to address applications that require much higher speed and dependability than CAN comes close to providing. This is mostly based on its applications in safety-critical systems as there is no room for failure. FlexRay is an excellent example of dual-communications line redundancy.

If a communications line becomes faulty, the backup communication line can immediately assume communications without any interruption of communication. The timing of message transmission is strictly time-triggered.

Speed: FlexRay is capable of transmitting up to 10 Mbit/s on each channel and providing absolutely guaranteed delivery of messages at the time they are scheduled — this is especially important in how systems control steering, brakes, etc.

Usage: Due to complexity and expense, it is only seen in high-end, luxury vehicles equipped with advanced 'drive-by-wire' systems (steering, brakes, etc. via electronic signals) [6].

■ Legacy Diagnostic Protocols

Before CAN became the standard, older vehicles utilised various methods for diagnostic purposes. One of the most prevalent methods pre-2008 was the K-Line (ISO 9141), which was the basis for various types of protocols such as KWP2000. CAN-based diagnostics (ISO 15765) replaced K-Line for OBD-II. The base design of K-Line has influenced the physical layer of the LIN protocol. If you've worked on an older vehicle, you may have encountered K-Line; however, it is obsolete for current model year vehicles.

■ 2.2 OBD-II Standard

■ 2.2.1 Overview and Standardization

OBD-II (On-Board Diagnostics Version Two) is a universal computer-based control system contained in all vehicles to manage the function of components that control the emissions from the engine. The Controller Area Network (CAN) provides the physical transmission medium for the data's movement, while OBD II establishes a higher-level definition of the protocol that determines the formatting, requesting and interpretation of diagnostic information [31].

■ Legislative Background and Intent

The creation of OBD systems is primarily attributed to the emissions control legislation coming out of California State's California Air Resources Board (CARB). The adoption of a standardized OBD-II was implemented in four different phases throughout regulatory regions:

1. All US manufactured (domestic) passenger automobiles and light duty trucks manufactured after January 1, 1996
2. All European manufactured gasoline powered automobiles beginning January 1, 2001 and diesel fuel automobiles beginning January 1, 2003 (EOBD)
3. Beginning with 2010 model year, all US manufactured heavy duty vehicles shall utilize the OBD-II system in compliance with the established OBD standards.

■ ISO 15765-4 OBD II Diagnostics using CAN Protocol

Since 2008, CAN bus communication (ISO 15765-4) has been the required signaling protocol for OBD II on all vehicles manufactured for sale in the United States. The ISO 15765-4 standard has also defined specific requirements for the underlying CAN Protocol (ISO 11898), including fixed CAN protocol (ISO 11898) bit rates of 250 kbit/s and a maximum length for diagnostic adapters.

2.2.2 OBD II Physical Connector

The SAE J1962, or ISO 15031-3, standard defines the physical connection between vehicles and diagnostic equipment. OBD II uses a 16-pin D-shaped female connector, which provides external access to this network. It's physically wired to the Central Gateway providing the connection point. This allows diagnostic tools to communicate through the Gateway to reach any ECU across all vehicle networks for troubleshooting and data monitoring.

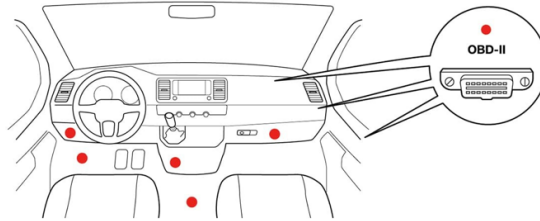


Figure 2.9: OBD-II most common locations [37]

OBD II Connector Pin-out

OBD II utilizes a connector that is designed to support legacy diagnostic protocols such as K-Line; however, pins 6, & 14 (ISO 15765 CAN High and CAN Low) are used only for new CAN diagnostics [8].

1. Pin 6 - CAN High (CAN_H)
2. Pin 14 - CAN Low (CAN_L)
3. Pin 16 - 12V Battery Power (or 24 V)
4. Pin 4 - Chassis Ground
5. Pin 5 - Signal Ground

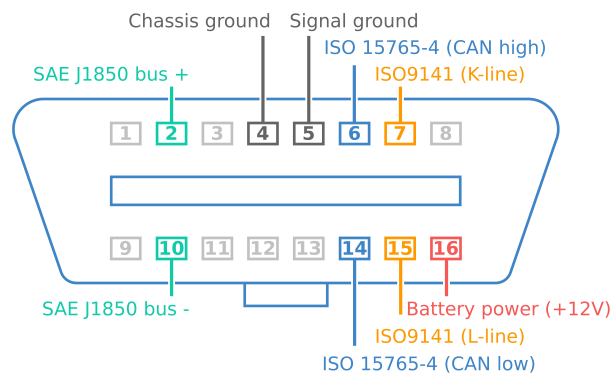


Figure 2.10: OBD-II Connector pinout [9]

2.2.3 Parameter IDs

OBD-II has a standardized format for messages as opposed to a proprietary CAN that uses a manufacturer specific DBC file (as mentioned in Section 2.1.4) to decode the messages and the standard for this is SAE J1979. PIDs (along with defined formulas) provide the means necessary to convert the raw data into physical values and thus vehicle specific databases are not required when accessing standard diagnostic data. PIDs represent standard identifiers used to request vehicle data from the vehicle ECUs through the OBD-II interface.

Standard vs Manufacturer-Specific PIDs

PIDs are categorized into two main types:

- **Standard PIDs (Mode 0x01):** These PIDs are specified by the SAE J1979 standard and are mandatory for a vehicle to meet OBD-II compliance by all manufacturers. The Standard PIDs encompass important, emission-related attributes of a vehicle, such as engine RPM, vehicle speed, coolant temperature, and fuel system status. The standard PIDs also provide consistency in the required format of the data across all vehicle manufacturers to allow for any generic scanning tool to retrieve basic diagnostic information from all compliant vehicles.
- **Manufacturer-Specific PIDs (Mode 0x09 and others):** In addition to the standard PIDs, manufacturers can add proprietary PIDs to their vehicles to provide additional vehicle-specific data. Each manufacturer has unique manufacturer-specific PIDs that may require specialized diagnostic software to interpret.

Common Standard PIDs

Table 2.2 lists frequently used PIDs with data byte formulas and ranges.

Table 2.2: Common OBD-II Standard PIDs

PID	Parameter	Formula	Unit	Range
0x0C	Engine RPM	$(A \times 256 + B)/4^2$	rpm	0–16 383
0x0D	Vehicle Speed	A	km h ⁻¹	0–255
0x05	Coolant Temp	$A - 40$	°C	–40 to +215
0x04	Engine Load	$A \times 100/255$	%	0–100
0x11	Throttle Position	$A \times 100/255$	%	0–100
0x0F	Intake Air Temp	$A - 40$	°C	–40 to +215
0x10	MAF	$(A \times 256 + B)/100$	g/s	0–655.35

² A and B denote the first and second bytes of the OBD-II response frame, respectively.

■ PID Decoding Examples

Example 1: Engine RPM (PID 0x0C). Engine RPM is transmitted using two data bytes (A and B). The formula to calculate RPM is:

$$\text{RPM} = \frac{(A \times 256) + B}{4} \quad (2.7)$$

Sample Response:

Suppose the ECU responds with the following CAN message for PID 0x0C:

Response: 04 41 0C 1A F8 00 00 00

Breaking down this response:

- 04 = Number of additional data bytes
- 41 = Mode \$01 response (0x40 + 0x01)
- 0C = PID (Engine RPM)
- 1A = Data byte A (26 in decimal)
- F8 = Data byte B (248 in decimal)

Calculation:

$$\text{RPM} = \frac{(26 \times 256) + 248}{4} = \frac{6656 + 248}{4} = \frac{6904}{4} = 1726 \text{ rpm} \quad (2.8)$$

Example 2: Vehicle Speed (PID 0x0D). Vehicle speed requires only one data byte (A), and the value is directly reported in km h^{-1} :

$$\text{Speed} = A \text{ km h}^{-1} \quad (2.9)$$

Sample Response:

Response: 03 41 0D 50 00 00 00 00

Breaking down this response:

- 03 = Number of additional data bytes
- 41 = Mode \$01 response
- 0D = PID (Vehicle Speed)
- 50 = Data byte A (80 in decimal)

Calculation:

$$\text{Speed} = 80 \text{ km h}^{-1} \quad (2.10)$$

This straightforward encoding makes vehicle speed one of the simplest PIDs to interpret.

2.3 Market Analysis of Existing OBD-II Solutions

An analysis of existing OBD-II diagnostic solutions reveals two primary product categories: tools focused on vehicle fault diagnostics and tools using the OBD-II interface primarily for tracking and telematics purposes [19].

OBDeleven

- **Key Features:** Provides deep access to proprietary systems with emphasis on VAG Group (VW, Audi, Škoda, Seat), BMW, and Toyota. Offers One-Click Apps for coding operations without manual programming.
- **Advantages:** Intuitive interface and high level of system integration.
- **Disadvantages:** Many features require additional credits or an annual subscription [38].



Figure 2.11: OBDeleven diagnostic dongle [22]

■ Generic ELM327 Adapters

Budget-level diagnostic dongles that act as a bridge between the vehicle ECU and external applications such as Torque or Car Scanner [11].

- **Key Features:** Provide access to DTC codes and live sensor readings via standard OBD-II PIDs.
- **Advantages:** Extremely affordable and compatible with virtually all vehicles manufactured after 2001.
- **Disadvantages:** Limited to the engine control unit only; the market is flooded with low-quality clones with connectivity issues.



Figure 2.12: Generic ELM327 Bluetooth adapter [17]

■ SIZZAPP — Signal Profi 4G GPS Tracker

Unlike diagnostic tools, SIZZAPP is designed primarily for vehicle protection and tracking, using the OBD-II port for power and minimal telemetry [29].

- **Key Features:** GPS tracker with 4G connectivity providing location history, trip records, and motion detection alarms for theft or collision events.
- **Advantages:** No monthly subscription fee required.
- **Disadvantages:** Focused on telematics rather than diagnostics; limited OBD-II data access.



Figure 2.13: SIZZAPP 4G GPS OBD tracker [30]

■ T-Mobile Smart Car

The Smart Car system represents the connected car direction, using the OBD-II port for telematics and in-car connectivity [33].

- **Key Features:** Integrated SIM card operating as a mobile hotspot, with GPS tracking and basic vehicle state monitoring.
- **Advantages:** Allows a Wi-Fi connection for car.
- **Disadvantages:** Requires a monthly subscription and provides very limited diagnostic data.



Figure 2.14: T-Mobile Smart Car OBD-II device [15]

Chapter 3

Design and Implementation of the Device

3.1 Introduction

The main goal of the thesis was to develop a device for analyzing real-time automotive data from the CAN bus using the OBD-II interface of the vehicle. In accordance with the set requirements, the system development process was divided into three major areas:

- designing a custom diagnostic device.
- development of embedded firmware.
- implementing a cloud-based server for remote data monitoring based on the use of a cellular module.

3.2 Hardware Design

3.2.1 System Requirements and Component Selection

The hardware design of the diagnostic device, based on four key functional blocks¹:

- a main microcontroller for system control and data processing.
- a CAN FD controller for communication with the vehicle bus.
- a cellular module for remote data transmission and vehicle positioning.
- a power supply subsystem adapted to the automotive environment.

The selection of each component was driven by the requirements of automotive operation and the need for compatibility with the OBD-II/CAN FD protocol. The following subsections describe each component and the rationale behind its selection.²

¹The complete schematic of the device is provided in Appendix B.

²Detailed datasheets of all components from this chapter are provided in Appendix A.

■ Microcontroller — ESP32-S3-MINI-1-N8

The ESP32-S3-MINI-1-N8 module by Espressif Systems was chosen as the main computing element of the device. The motivation behind the decision included the small size of the module, the presence of the PCB 2.4 GHz Wi-Fi antenna, and capabilities of the dual-core 32-bit Xtensa LX7 CPU with 240 MHz frequency. Among the most important characteristics, the SPI, I²C, and UART interfaces enable peripheral connectivity. The main electrical specifications can be found in Table 3.1.

Table 3.1: Key features of the ESP32-S3-MINI-1 module (N8 variant)

Parameter	Specification
MCU core	Dual-core Xtensa [®] LX7, 32-bit, up to 240 MHz
On-chip memory	384 KB ROM, 512 KB SRAM, 16 KB RTC SRAM
External flash	8 MB Quad SPI
Wireless connectivity	Wi-Fi 802.11 b/g/n (2.4 GHz), Bluetooth 5 LE
GPIO pins	39 (multiplexed with peripherals)
Key interfaces	SPI, UART, I ² C, I ² S, USB 2.0 OTG, TWAI [®] , ADC
Integrated peripherals	TWAI [®] (CAN 2.0), touch sensor, temperature sensor, LED PWM, RMT
Operating voltage	3.0 V–3.6 V
Operating temperature	−40 °C to +85 °C
Antenna option	External connector (U.FL/IPEX)
Module dimensions	15.4 mm × 15.4 mm × 2.4 mm
Package	65-pin module

■ CAN FD Controller — MCP2518FD

An external MCP2518FD CAN FD controller by Microchip Technology was used for vehicle CAN bus communication. While the ESP32-S3 includes a built-in TWAI controller, it only supports CAN 2.0 at speeds up to 1 Mbit/s and requires an external transceiver. The MCP2518FD provides full CAN FD support (ISO 11898-1:2015) with data phase speeds up to 8 Mbit/s and backward compatibility. Communication with the ESP32-S3 is managed via the SPI bus at up to 20 MHz. Key features include:

- 31 configurable FIFO buffers for data transmission and reception.
- 32 configurable message acceptance filters.
- Automatic CRC check during SPI transfers to prevent data corruption.
- Automotive qualification with operating temperatures from −40 °C to +125 °C.

■ LTE & GPS Module — SIMCom A7670E

The SIMCom A7670E module enables remote data transmission and positioning via LTE Cat-1 with GSM/GPRS fallback. It integrates a GNSS receiver, eliminating the need for external GPS hardware, and communicates with the ESP32-S3 via UART using AT commands. The module's 5 V–10 V supply range was integrated into the power subsystem design. Key specifications include:

- LTE Cat-1 connectivity with 2G fallback for limited coverage areas.
- Integrated TCP/IP and HTTP(S) stacks for direct server communication.
- GNSS engine for vehicle position and time synchronization.
- Low-power modes suitable for permanent vehicle installation.

LTE connectivity is provided via a microstrip PCB antenna with a gain of 5 dBi and a U.FL connector.

■ Power Supply — TPS54331

The device is powered directly from the vehicle OBD-II connector, which provides a nominal voltage of 12.2–12.7 V. During engine operation, the voltage range could be between 13.5 V and 14.7 V. To regulate this input to a stable output suitable for the system components, the TPS54331 synchronous step-down converter from Texas Instruments was selected. This device integrates a power MOSFET and is capable of delivering up to 3 A of continuous output current. The output voltage was set to 5 V by means of a resistor divider connected between the output and the feedback node. The converter operates at a fixed switching frequency of 570 kHz, which reduces the physical size of the external inductor. An operating temperature range of -40°C to $+125^{\circ}\text{C}$ ensures reliable performance across the full range of conditions expected in an automotive environment.

3.2.2 Circuit Design

The circuit architecture is composed of several functional blocks, as shown in Figure 3.1.

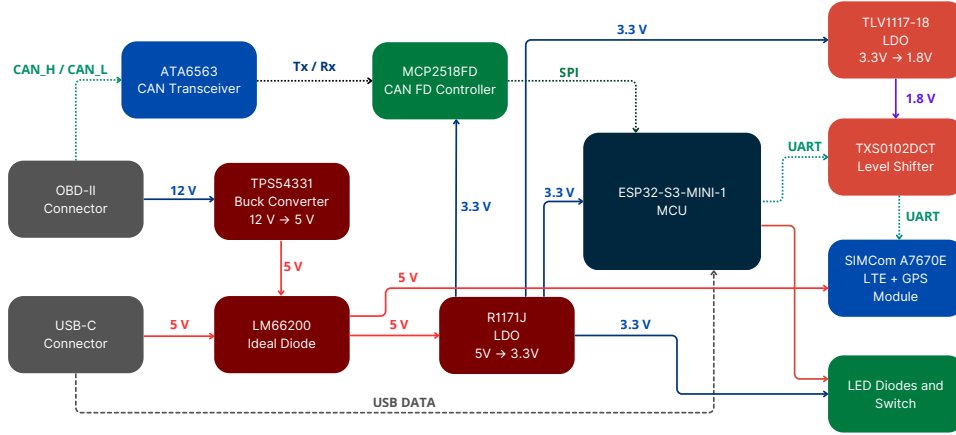


Figure 3.1: CAN Bus Sniffer block diagram

OBD-II Connector

The OBD-II connector provides power and CAN bus connection from the vehicle. Both CAN_H and CAN_L lines have implemented ESD protection on PCB. There is an option to add impedance termination of the bus. This application consists of two $60\ \Omega$ resistors and one $4.7\ \text{nF}$ capacitor (marked DNP in the BOM file) in case there is only one device attached to the bus, thus maintaining the impedance of $120\ \Omega$ required by the CAN protocol. The $0\ \Omega$ resistor allows selection of either signal ground (pin 5) or chassis ground (pin 4). In this design, signal ground was preferred, because it provides higher noise immunity from vehicle electrical system.

■ DC/DC Step-down Converter — TPS54331

The step-down DC/DC converter is connected to the power input of the OBD-II connector and transforms the vehicle supply voltage to 5 V. The regulated output voltage is determined by the resistor divider on the feedback pin of the IC according to Equation 3.1:

$$\begin{aligned} V_{out} &= 0.8 \times \left(1 + \frac{R_1}{R_2} \right) \\ &= 0.8 \times \left(1 + \frac{10 \text{ k}\Omega}{1.91 \text{ k}\Omega} \right) \\ &= 4.99 \text{ V} \end{aligned} \tag{3.1}$$

where V_{out} is the regulated output voltage, $R_1 = 10 \text{ k}\Omega$, and $R_2 = 1.91 \text{ k}\Omega$, yielding a nominal output voltage of 5 V.

A $6.8 \mu\text{H}$ output inductor was selected with respect to the 570 kHz switching frequency. Input protection consists of a Schottky diode, an ESD diode, a 2 A polyfuse, and a 120Ω ferrite bead, which suppresses high-frequency noise in both directions between the vehicle network and the converter.

■ USB-C Connector

Two $5 \text{ k}\Omega$ resistors on CC1 and CC2 identify the device as a USB sink. A 2 A polyfuse and ESD protection are included on the VBUS line. The USB-C input is routed to the LM66200 ideal diode, allowing the device to be powered from USB during development without a vehicle connection.

■ Ideal Diode — LM66200

The LM66200 compares the OBD-II and USB-C input voltages and automatically connects the higher source to the output rail, blocking reverse current from both inputs. This eliminates the need for manual power source selection and avoids the forward voltage drop of conventional signal diodes.

■ LDO Regulator — R1171J

The R1171J converts the 5 V supply rail to 3.3 V required by the ESP32-S3, MCP2518FD, status LEDs, and the LDO for the SIMCom module. Two ceramic capacitors of $4.7 \mu\text{F}$ ensure stability across the complete load range.

■ Level Shifter — TXS0102DCT and TLV1117LV

The SIMCom A7670E module communicates via UART, which operates at 1.8 V, while the ESP32 uses 3.3 V logic. A TXS0102DCT bidirectional shifter translates these voltages without direction control. Its 1.8 V supply is derived from the 3.3 V rail via a TLV1117LV LDO regulator.

■ CAN Bus Interface — MCP2518FD and ATA6563

The MCP2518FD CAN FD controller processing data from vehicle and handles the CAN protocol stack operations. The communication between the MCP and the ESP32 is performed through the SPI bus interface. The controller communicates with the ATA6563 transceiver via TX/RX signals, which translates into differential CAN_H and CAN_L signals on the physical bus layer. The 3.3 V logic level of the MCP2518FD is also applied to the VIO pin of the ATA6563, setting its logic threshold accordingly.

■ User Interface — LEDs and Buttons

Three status LEDs indicates device activity.

- **LED1 (red) — LTE/POST status:** lights up when the modem is connected, flashes quickly once for each successful POST, turns off in case of an error
- **LED2 (blue) — GPS:** flashes 3 times + pauses until a fix is acquired; stays lit when a fix is acquired
- **LED3 (green) — CAN activity:** flashes with every received CAN frame.

A power LED is connected directly to the 3.3 V rail. A Reset button is connected to the EN pin and a Boot button to GPIO0, enabling firmware download mode when held during reset. User button is not implemented yet and could be used for future application.

■ 3.2.3 PCB Design

- The PCB was designed as a four-layer board with measurements of 50×45 mm and 1.6 mm thickness. The schematic includes two external signal layers and a continuous ground layer at the second layer. The 5 V power rail is routed as a trace on the bottom layer, while the 3.3 V rail is implemented as a local copper pour in areas where it is required, rather than a continuous plane across the entire board. PCB was manufactured by Printed Mělník.³
- The ground plane provides a low-impedance return path for all signals and suppresses electromagnetic interference across the board. Via stitching ensures electrical continuity between ground pours on all layers, with increased via density around the ESP32-S3-MINI-1 module to reduce ground impedance in the Wi-Fi and Bluetooth frequency range.
- Special attention was paid to the layout of the TPS54331 switching converter. The output capacitors C18, C19, C20 and the output inductor, and the ESD diode were placed as close as possible to the IC, minimising the area of the high-frequency switching current loop and reducing radiated electromagnetic interference (as shown in Figure 3.3). Six thermal vias under the exposed pad conduct heat to the inner ground plane.
- The SIMCom GSM/GNSS module is integrated as a standalone breakout board connected to the main PCB via a 7-pin header. The header carries the following signals: UART RX, UART TX, VDD power supply, PWRKEY for module power control, Sleep pin for low-power mode control, and two GND pins, as detailed in Table 3.2. This approach allows the module to be replaced or disconnected independently of the main board. External LTE and GPS antennas are connected to the module via dedicated U.FL connectors. It has also 2 slots for Micro SIM card.

Whole PCB layout is shown in Figure 3.2.

³Printed — plošné spoje a osazování, <https://printed.cz/>

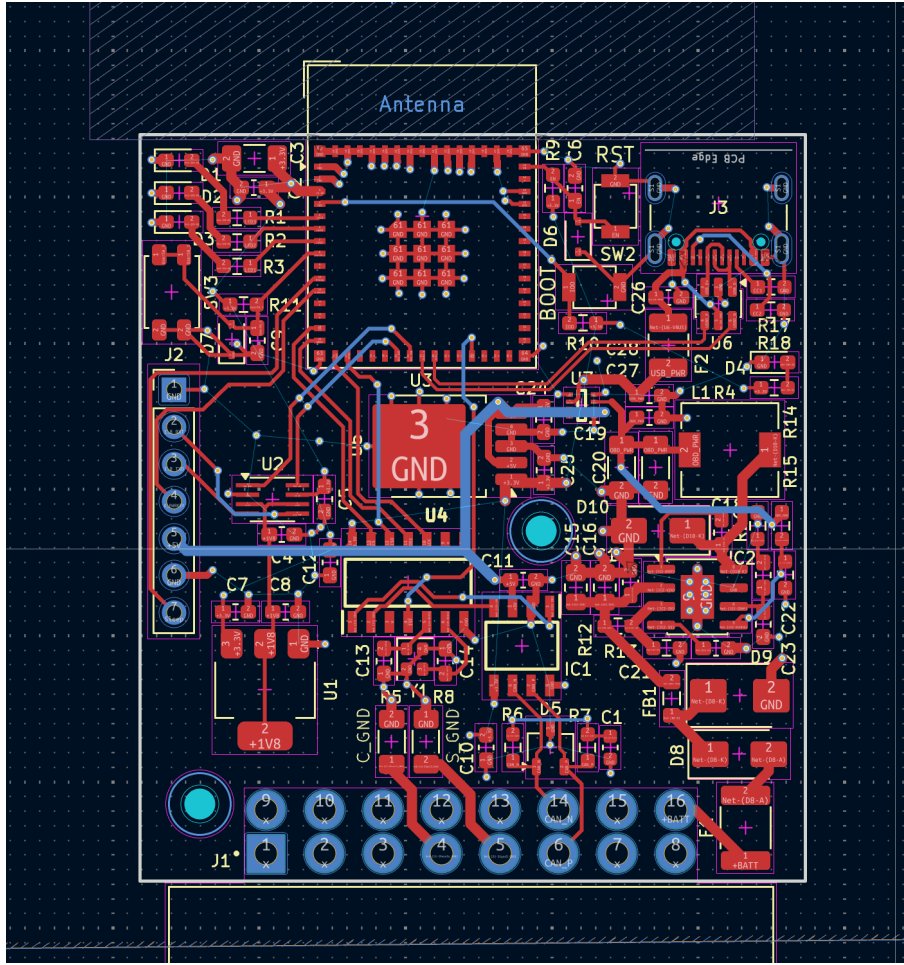


Figure 3.2: PCB layout of 4-layer board

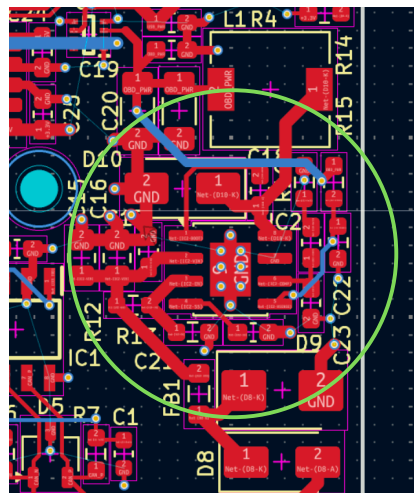


Figure 3.3: PCB layout showcasing the short high-frequency current loop path

3.2.4 System Pin Assignment

Table 3.2 summarises the GPIO pin assignment of the ESP32-S3 module grouped by functional block.

Table 3.2: ESP32-S3 GPIO Pin Assignment by Functional Block

Pin (IO)	Signal	Function / Destination	Component
3V3	+3V3	ESP32-S3 power supply	Power
IO1	LED3	Status indicator LED 3	Red LED
IO2	LED2	Status indicator LED 2	Blue LED
IO3	LED1	Status indicator LED 1	Green LED
IO4	USER_SW	User button	Button
EN	RESET_BTN	Reset button	System button
IO0	BOOT_BTN	Boot button	System button
IO6	SLEEP	Sleep mode control pin	SIMCom module
IO7	PWRKEY	Module power on/off button	SIMCom module
IO10	CS	SPI Chip Select	MCP2518FD
IO11	MOSI	SPI Data (Master Out Slave In)	MCP2518FD
IO12	SCK	SPI Clock	MCP2518FD
IO13	MISO	SPI Data (Master In Slave Out)	MCP2518FD
IO17	RXD	UART Receive	SIMCom module
IO18	TXD	UART Transmit	SIMCom module
IO23	USB_D−	USB differential pair (negative)	USB-C
IO24	USB_D+	USB differential pair (positive)	USB-C
IO47	INT	Interrupt output	MCP2518FD

■ 3.3 Firmware Development

■ 3.3.1 Software Architecture

The firmware was developed using the PlatformIO development environment, which provides dependency management and integrated serial monitoring. Two functional variants of the firmware were implemented. The first variant transmits vehicle data to the cloud server.

In the second variant, the ESP32-S3 functions as a local Wi-Fi access point and serves a real-time dashboard to the connected device directly.

The firmware is organized into the following functional modules:

- **CAN communication** — MCP2518FD initialization, OBD-II request transmission and response reception via the ACAN2517FD library
- **OBD-II decoder** — PID request construction, response parsing and value calculation according to SAE J1979
- **LTE transmission** — AT command sequences for network registration and HTTPS POST to the remote server
- **GPS acquisition** — GNSS position reading via AT commands with fallback logic when no fix is available
- **Web server** — local Wi-Fi access point serving a real-time dashboard over HTTP, used in the standalone variant
- **LED indication** — visual feedback on CAN activity, LTE connectivity and GPS fix status

■ 3.3.2 System Flow Diagram

During initialization, the system configures all peripherals sequentially — SPI, CAN controller, LTE module, and GNSS receiver. Network registration is confirmed via `AT+CREG?` returning 0,1 or 0,5.

The core firmware architecture is governed by a Finite State Machine (FSM) that manages power consumption and operational modes based on CAN bus activity. The FSM transitions between four primary states: `BOOT`, `DRIVING` (active communication), `IDLE_AWAKE` (engine off, ECU quiet), and `LIGHT_SLEEP` (power-saving mode).

In the active `DRIVING` state, the main loop operates entirely asynchronously to prevent data loss. A GNSS position is continuously polled using `AT+CGNSSINFO` and stored if a valid fix is available. OBD-II requests are dispatched sequentially via the broadcast identifier `0x7DF`, Mode `0x01`. Rather than utilizing blocking timeouts, the system sends requests at strictly defined intervals — 50 ms for critical dynamic data (e.g., RPM, speed) and 150 ms for slow-changing parameters. Responses from the vehicle's ECUs, expected

in the range from 0x7E8 to 0x7EF, are processed continuously via a non-blocking CAN service routine.

Data transmission is decoupled from the OBD-II polling cycle. A JSON payload is sent to the remote server via HTTPS POST at a fixed interval of 2000 ms. To optimize bandwidth and processing, the payload alternates between a smaller "fast" subset and a "full" telemetry set every third cycle. Receipt of an HTTP 200 OK denotes successful delivery, which is visually reflected by the LTE status LED. To ensure high reliability, if three consecutive HTTP transmissions fail, the microcontroller triggers a hardware reset of the modem via the PWRKEY sequence and initiates a full network reconnection.

Firmware execution and the state transitions are illustrated in Figure 3.4.

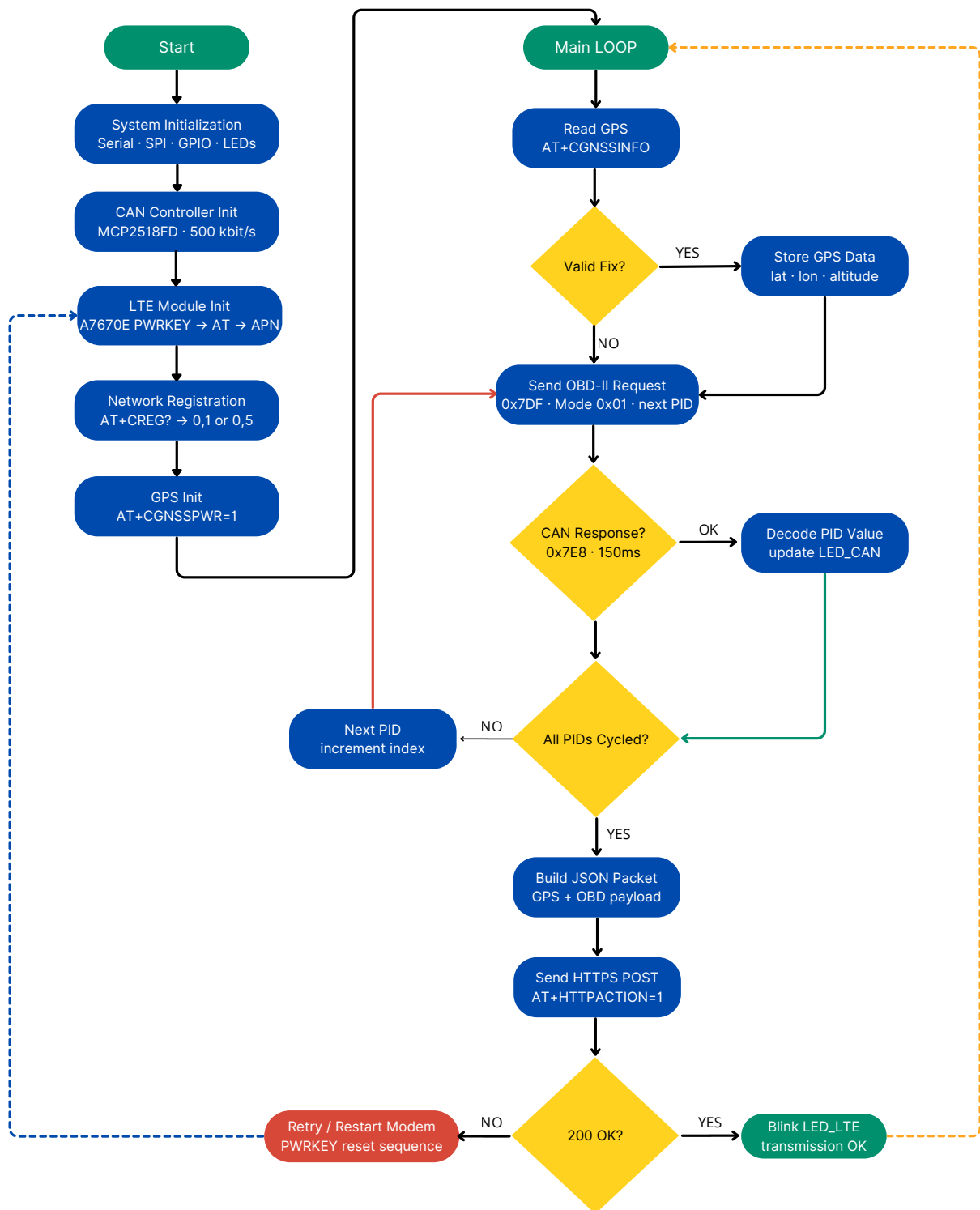


Figure 3.4: Main loop flow diagram

3.3.3 CAN Controller Initialization

The initialization of the MCP2518FD CAN controller over SPI is managed using the ACAN2517FD library. The SPI interface was configured with the pins mentioned in Table 3.2. The controller was initially configured to operate under CAN 2.0 with a nominal bit rate of 500 kbit/s, which ensures compatibility with the CAN 2.0B protocol used by the vehicle ECUs. The chosen bit rate corresponds to the ISO 15765-4 standard applicable to most present-day vehicles.

In its default hardware configuration, the MCP2518FD controller silently discards all incoming frames unless acceptance filters are configured. These filters can be explicitly defined in software configuration, using methods such as `appendFilter()` to specify exactly which CAN IDs or ranges the hardware should forward to the microcontroller, thereby preventing CPU overload. However, this project utilizes the library's `appendPassAllFilter()` method (applied implicitly during initialization) to forward all CAN traffic. While processing all messages in software could overwhelm weaker processors, the ESP32-S3 effortlessly handles the load required for scanning broad OBD-II data thanks to its dual-core architecture and high clock rate.

```

1  ACAN2517FDSettings canSettings(
2      ACAN2517FDSettings::OSC_40MHz,
3      500 * 1000,
4      DataBitRateFactor::x1
5  );
6  canSettings.mRequestedMode = ACAN2517FDSettings::Normal20B;
7  const uint32_t canErr = can.begin(canSettings, canISR);
8  if (canErr == 0) {
9      Serial.println("[CAN] OK @ 500 kbit/s");
10     can.appendPassAllFilter();
11 } else {
12     Serial.print("[CAN] ERROR: 0x");
13     Serial.println(canErr, HEX);
14 }
15 \caption{Initialization of the MCP2518FD CAN controller at
16     500 kbit/s}
17 \label{lst:can_init}

```

Listing 3.1: Initialization of the MCP2518FD CAN controller at 500 kbit/s

3.3.4 OBD-II PID Decoding

OBD-II diagnostic data is requested using the standard broadcast identifier 0x7DF with Mode 0x01, as defined by SAE J1979. Prior to starting cyclic reading, a PID support scan is performed by requesting PIDs 0x00, 0x20, 0x40, and 0x60, which return 32-bit data frames indicating which parameters are supported by the connected vehicle. Once the supported parameters are known, actual sensor data is periodically requested. The received raw data bytes are then passed to the `decodePID()` function, which applies the appropriate formula based on the PID identifier to calculate the final physical values.

```

1 void sendOBDRequest(uint8_t pid) {
2     CANFDMessage frame;
3     frame.id = 0x7DF;
4     frame.len = 8;
5     frame.data[0] = 0x02; // Number of additional data bytes
6     frame.data[1] = 0x01; // Mode 01 (Current Data)
7     frame.data[2] = pid; // Requested PID
8     memset(&frame.data[3], 0, 5); // Padding remaining zero
   bytes
9
10    can.tryToSend(frame);
11 }
12
13 float decodePID(uint8_t pid, uint8_t a, uint8_t b) {
14     switch (pid) {
15         case 0x0C: return ((a * 256.0f) + b) / 4.0f; // RPM
16         case 0x05: return a - 40.0f; //
   Coolant Temp
17         case 0x0D: return a; //
   Speed
18         case 0x11: return (a * 100.0f) / 255.0f; //
   Throttle
19         default: return a;
20     }
21 }

```

Listing 3.2: OBD-II request transmission and PID decoding functions

3.3.5 LTE Communication

The SIMCom A7670E module is controlled via AT commands transmitted over UART at 115200 bps. The power-on process involves a pulse on the PWRKEY pin to initialize the module, followed by a five-second boot delay. Network registration state is verified via the `AT+CREG?` command; responses of 0,1 or 0,5 signify successful registration. Signal strength is evaluated via `AT+CSQ`; test results usually indicated a signal strength value of 28, which corresponds to approximately -57 dBm and is considered a strong signal⁴. APN configuration for the O2 CZ network is performed via `AT+CGDCONT`. This is done by sending the `AT+CGDCONT=1,"IP","internet"` command, to which the module responds with OK. Once network connection is established, an HTTPS POST request is sent to the remote server via the following command sequence:

```

1 AT+CSSLCFG="sslversion",0,3           // TLS 1.2
2 AT+CSSLCFG="authmode",0,0             // no cert verification
3 AT+CSSLCFG="enableSNI",0,1           // SNI required by
   Railway
4 AT+CSSLCFG="ignorelocaltime",0,1
5 AT+HTTPINIT
6 AT+HTTPPARA="URL","https://server/data"
7 AT+HTTPPARA="CONTENT","application/json"
8 AT+HTTPPARA="USERDATA","Authorization: Bearer <token>"
9 AT+HTTPPARA="SSLCFG",0
10 AT+HTTPDATA=<length>,10000
11 AT+HTTPACTION=1                       // +HTTPACTION: 1,200,<
   len>
12 AT+HTTPTERM

```

Listing 3.3: AT command sequence for HTTPS POST transmission

A critical finding during testing was that Railway's edge infrastructure requires the TLS SNI (Server Name Indication) extension to be explicitly enabled via `AT+CSSLCFG="enableSNI",0,1`. Without this parameter, the module firmware A131B03 consistently returned error code 715 (TLS handshake failure), as the server could not determine which certificate to present for the multi-tenant domain `*.up.railway.app`. The `AT+HTTPSSL=1` command, present in earlier firmware versions, is not supported by this firmware build and must be replaced by the `AT+HTTPPARA="SSLCFG",0` parameter referencing the pre-configured SSL profile. The remote server is deployed on Railway, a cloud Platform-as-a-Service (PaaS); its architecture and API design are described in detail in Section 3.4.

⁴CSQ values are in the range from 0 to 31, when 31 is considered as the strongest signal.

3.3.6 GPS Acquisition

GPS data are acquired by the `AT+CGNSSINFO` command, which returns latitude, longitude, altitude, and speed if a fix is available. Two acquisition modes are recognised:

- **Cold start** — no previous position or time information is available; the process may take several minutes to acquire an initial fix.
- **Hot start** — cached ephemeris and almanac data are utilized to achieve a fix within seconds⁵.

When a valid fix cannot be acquired, the GPS parameters are excluded from the JSON packet and system does not visualize location data.

A fix refers to the moment when the GNSS receiver successfully receives signals from a sufficient number of satellites — at least four — and is able to calculate a valid position estimate.

A high-gain GNSS antenna with a gain of 28 dBi with the U.FL connector is used. It supports GPS at 1575 MHz, BeiDou at 1561 MHz, and GLONASS at 1602 MHz, with an impedance of 50 Ω .

In case of a weak signal, and a situation where the antenna is not able to get GPS data, the data are not visualized, but it is still trying to get a fix.

For future implementation, usage of Cell ID could be highly beneficial. This would significantly enhance location tracking precision and serve as a fall-back mechanism in areas with poor GPS coverage, depending on the capabilities of the mobile network operator



Figure 3.5: Illustration photos of LTE antenna (left)^[34] and GNSS antenna (right)^[36]

⁵Ephemeris data contains precise short-term satellite positions, while the almanac provides long-term constellation orbits. Caching this information allows the receiver to acquire a rapid fix without re-downloading it from the satellites.

3.4 Cloud Infrastructure

3.4.1 Deployment Platform

The **Railway.app**⁶ platform was selected as the deployment environment, a Platform-as-a-Service (PaaS) provider that enables continuous cloud deployment directly from a GitHub repository. When changes are pushed to the repository, Railway automatically rebuilds and redeploys the application without any manual server administration. The platform provides a fixed public HTTPS endpoint, which serves as the destination for all data transfers initiated by the ESP32-S3 device. The back-end is implemented in Python using the **Flask** framework.

3.4.2 REST API Design

The server exposes a REST API to manage the data flow between the IoT device and the web dashboard. The core endpoints are summarised in Table 3.3.

Table 3.3: REST API Endpoints

Method	Endpoint	Description
POST	/data	Receives a JSON payload from the ESP32-S3, verifies the nonce, updates the RAM cache, and stores it in the database
GET	/data/latest	Returns the most recent merged state of all parameters; polled by the dashboard at 5 Hz
GET	/data	Returns stored data packets for historical view
POST	/command	Accepts a command string and places it in a queue for retrieval by the device

An example of a JSON packet transmitted by the ESP32-S3 to the endpoint is shown below. The firmware transmits semantic string keys directly (e.g., "rpm", "speed"). This ensures a minimal payload size:

```

1 {   "nonce": 1714928192345,
2     "type": "full",
3     "rpm": 2500.0,
4     "speed": 80.0,
5     "throttle": 45.0,
6     "engineLoad": 35.0, }
```

Listing 3.4: JSON payload structure transmitted to the remote server

⁶<https://railway.app>

On the server side, no strict database schema or decoding dictionary is enforced. Instead, the PostgreSQL database utilizes a `JSONB` (Binary JSON) column for flexible storage. This approach allows the data structure to be extended with additional OBD-II parameters in the future without requiring any database schema migrations.

To optimize performance for the live web dashboard, the server implements an in-memory state caching mechanism (`state_cache`). Because the microcontroller alternates between sending smaller "fast" packets and occasional "full" packets to save bandwidth, the server merges incoming data into this cache⁷. The dashboard client (`data.js`) polls the `GET /data/latest` endpoint every 200 ms, receiving a seamlessly updated, complete snapshot of the vehicle's state without generating heavy database queries.

■ 3.4.3 Data Storage

The collected data packets are persistently stored in a PostgreSQL database hosted via the Railway platform. Each record in the `packets` table consists of a server-generated `timestamp`, an optional `event` string, and the telemetry payload stored in a `JSONB` (Binary JSON) format. Utilizing the `JSONB` data type allows for a flexible, schema-less approach, enabling the system to store varying sets of OBD-II parameters without requiring complex database migrations or predefined column structures.

PostgreSQL was selected for its robust persistence and advanced indexing capabilities. The connection between the Flask application and the database is secured via an encrypted connection string stored as an environment variable. `GET /data` endpoint is configured to retrieve the most recent history by default, while the database remains a complete, long-term archive for all telemetry data.

■ 3.4.4 Repository

The complete project is stored in a public GitHub repository. The repository structure is described in Appendix E.1.

https://github.com/Kubakl11/CAN_Bus_Sniffer_BP

⁷The "fast" packet consists of RPM, speed, engine load, and throttle position, while the "full" packet contains all supported PIDs and GNSS coordinates

3.5 Security Analysis

Since two independent prototypes were implemented, two different security approaches were made. The primary configuration utilises a cellular LTE module for remote data transmission to a cloud server, while the secondary configuration operates the ESP32-S3 as a local Wi-Fi access point, allowing the user to connect directly via a captive portal. Introducing wireless communication in both configurations — whether over a public cellular network or a local access point — introduced security risks that needed to be considered and resolved.

- **LTE configuration** — unauthorised access to the server API endpoints and replay attacks, in which a previously captured valid request could be retransmitted to inject false data into the server.
- **Local Wi-Fi AP configuration** — unauthorised network access, where without link-layer protection any nearby device could join the access point and interact with the web dashboard.

3.5.1 LTE Channel Security

The primary configuration utilises a cellular LTE module for remote data transmission. Three security measures were implemented to protect this communication channel.

Transport-Layer Encryption

Transport-layer encryption is ensured by communicating exclusively over HTTPS on port 443, with TLS (Transport Layer Security) termination handled by the Railway infrastructure:

```

1 AT+HTTPINIT
2 AT+HTTPPARA="URL","https://server.railway.app/data"
3 AT+HTTPPARA="SSLCFG",0
4 AT+HTTPACTION=1          // response: +HTTPACTION 1,200,len
5 AT+HTTPTERM

```

Listing 3.5: AT command sequence for HTTPS POST request

To maintain robustness across device reboots, the server tolerates a nonce decrease exceeding 10^9 , which indicates that the ESP32-S3 has restarted with a lower boot counter value — for example after a firmware reflash or NVS clear — and resets the accepted baseline accordingly. In all other cases, a non-increasing nonce is rejected as a replay attempt.

```

1 if nonce <= last_nonce:
2     return jsonify({"error": "replay detected"}), 400
3 last_nonce = nonce

```

Listing 3.6: Server-side replay attack detection**Bearer Token Authentication**

Authorisation is provided using the Bearer token authentication method. The secret token is generated using a cryptographically strong random number generator and stored exclusively as a Railway environment variable on the server side and as a compile-time constant in the ESP32-S3 firmware:

```
1 SECRET_TOKEN = os.environ.get("SECRET_TOKEN", None)
```

Listing 3.7: Bearer token retrieval from environment variable

```
1 #define API_TOKEN "generated_token"
```

Listing 3.8: Bearer token definition in ESP32-S3 firmware

HTTP POST requests include an **Authorization** header carrying the token value, which is validated against the server-side environment variable. If the token is missing or invalid, the request is rejected with HTTP status 401 Unauthorized:

```
1 def check_auth(req):
2     if SECRET_TOKEN is None:
3         return False
4     auth = req.headers.get("Authorization", "")
5     return auth == f"Bearer {SECRET_TOKEN}"
```

Listing 3.9: Server-side Bearer token validation**Replay Attack Prevention**

Replay attacks are prevented by including a **nonce** field in every payload, derived from a persistent boot counter stored in non-volatile memory combined with the device uptime via `millis()`, ensuring the nonce strictly increases across device reboots. The server rejects any request where the nonce does not strictly exceed the previously accepted value:

```
1 if nonce <= last_nonce:
2     return jsonify({"error": "replay detected"}), 400
3 last_nonce = nonce
```

Listing 3.10: Server-side replay attack detection

■ 3.5.2 Local Wi-Fi Channel Security

The secondary configuration implements the ESP32-S3 serving as a local Wi-Fi access point, providing direct connectivity through a captive portal. Two security measures were implemented for this configuration.

Firstly, network access is secured using **WPA2-PSK** encryption, in which the ESP32-S3 soft AP acts as a router. Only users with the correct password are permitted to join the network, and all communication between the client and the device is protected by link-layer encryption.

```
1 WiFi.softAP(WIFI_SSID, WIFI_PASSWORD);
```

Listing 3.11: WPA2-PSK access point initialization

Second, a captive portal was implemented using a DNS server running directly on the ESP32-S3. Once the client connects to the access point, the operating system automatically opens the dashboard interface without requiring the user to manually enter the device IP address.

```
1 dnsServer.start(DNS_PORT, "*", apIP);
```

Listing 3.12: Captive portal DNS server initialization

Since the access point operates as a standalone local area network with no internet connectivity, link-layer WPA2 encryption is considered sufficient for this configuration.

The developed web interface serves as the primary visualization tool for vehicle telemetry. Layout is inspired by modern automotive infotainment systems, focusing on quick readability of all critical parameters. The application can be served either directly from the ESP32-S3 acting as a standalone web server, or hosted externally on a remote server.

HTML, CSS, and JavaScript are used exclusively in the development of the front-end application. This approach ensures optimal performance in terms of resource usage, since the interface needs to run on the ESP32-S3. Time series charts are created using the open-source library **Chart.js**, enhanced by the **chartjs-plugin-zoom** plugin, which enables users to zoom and pan the chart interactively. GPS functionality is implemented using **Leaflet.js** with map tiles rendered from the Mapbox API. Data are gathered using a polling method with a 200 ms interval (5 Hz), ensuring a sufficient sampling rate for the presented parameters.

The dashboard is divided into six dedicated screens accessible through a bottom navigation bar, summarised in Table 3.4.

Table 3.4: Dashboard Screen Overview

Screen	Description
Cockpit	Animated radial gauges for engine RPM and vehicle speed, live sparklines and widgets summarising alarms and fuel consumption.
Vehicle Overview	Structured view of all available PID values categorised into logical groups (basic readings, temperatures, fuel system, engine...) with a tabular history log.
Live Graphs	Time series plots of selected sensor outputs with detailed inspection of trends or anomalies.
Fuel Consumption	Instantaneous fuel consumption alongside average counters (since start, since refuel, long-term) and a trend graph.
GPS Map	Real-time vehicle position tracking with GPX route export.
Alarms	Configurable threshold limits for critical parameters; exceeding a threshold triggers a visual warning.

Detailed screenshots of all screens are provided in Appendix C.



Figure 3.6: Main Cockpit screen featuring live radial gauges and summary widgets

■ 3.6.2 Advanced Diagnostics Interface

Beyond the primary scope of this thesis, an advanced diagnostic web interface was developed to serve as a visual and functional foundation for future implementations of remote ECU flashing. However, practical ECU flashing remains a highly complex task requiring vehicle-specific data, as each manufacturer implements proprietary security mechanisms — component protection, virtual component handshakes, seed-and-key authentication, and extensions to the standard UDS protocol. Full ECU programming support therefore remains a subject for future development.

Access to the interface is restricted by a login screen, since it allows direct interaction with the CAN bus and transmission of UDS (Unified Diagnostic Services) queries to the ECU. Session state is stored in the browser via `localStorage`, which is sufficient for the prototype.

The diagnostic view provides three tools:

- **Frame Builder** — manual construction and periodic transmission of CAN frames with CAN FD support, bit-rate switching (BRS), and an interactive bit matrix for payload manipulation.
- **Live Monitor** — real-time CAN traffic display with filtering and log export.
- **UDS & Coding** — constructs and dispatches ISO-TP requests for reading data identifiers (DIDs), clearing diagnostic trouble codes (DTCs), and switching diagnostic sessions.

Screenshots of the login screen and diagnostic interface are provided in Appendix C.

■ 3.7 Testing and Validation

■ 3.7.1 End-to-End Transmission Test

The primary objective of this test was to verify the complete data path from the ESP32-S3, through the SIMCom A7670E LTE modem, across the cellular network, to the Flask application on Railway.app, and finally to the SQLite database. The test isolates the transport layer from the CAN bus and the OBD-II decoder.

■ The Methodology

For testing purposes, dummy data were generated using the `random()` function. Each payload contains typical data points (engine RPM, vehicle speed, coolant temperature, throttle, fuel percentage, MAF, system voltage, and GPS location) along with a monotonically increasing nonce derived from `millis()`. The firmware initiates registration in LTE-only mode using `AT+CNMP=38` and automatically falls back to an alternative radio access technology if no LTE cell is detected within 30 seconds. An HTTPS POST request is sent via the command sequence `AT+HTTPINIT`, `AT+HTTPPARA`, `AT+HTTPDATA`, and `AT+HTTPACTION` on UART1. The Bearer token is carried in the `USERDATA` HTTP header parameter.

■ Results

The test ran for approximately 90 seconds without interruption, during which 18 packets were transmitted. All transmissions received HTTP 200 responses, confirmed by the modem prompt `+HTTPACTION: 1,200,16` and a response body containing the status field set to "ok". Network registration on LTE was achieved on the first attempt. The signal strength reported by `AT+CSQ` was 24 (−65 dBm). The serving network identified by `AT+COPS?` was O2 Czech Republic (PLMN 23002) on E-UTRAN.

An independent read-back via `GET /data` confirmed a strictly monotonically increasing sequence of nonce values with consistent timestamps. The total latency of one HTTPS POST transaction, measured between `AT+HTTPINIT` and the `+HTTPACTION: 1,200` notification, was approximately 3 seconds, including DNS resolution, TLS handshake, request transmission, and server-side processing. The corresponding console log is shown below:

```
[INIT] Phase 1: trying LTE-only (CNMP=38)...
[AT] AT+CREG? -> +CREG: 0,1 OK
[INIT] Registered on LTE!
[AT] AT+COPS? -> +COPS: 0,2,"23002",7
[AT] AT+CSQ -> +CSQ: 24,99
[INIT] Modem ready for HTTP communication
[HTTP] JSON: {"nonce":14479,"rpm":2267,"speed":63,
             "coolant_temp":91,"throttle":40,"voltage":13.9,
             "lat":50.068001,"lon":14.430600}
[AT] AT+HTTPINIT -> OK
[AT] AT+HTTPACTION=1 -> +HTTPACTION: 1,200,16
[AT] AT+HTTPREAD=0,512 -> {"status":"ok"}
[STAT] Total: 18 OK: 18 FAIL: 0
```

Listing 3.12: End-to-end HTTPS POST transaction log

3.7.2 Vehicle CAN Communication

In order to confirm the end-to-end functionality of the proposed system with regard to data collection, an actual test drive was performed using a real vehicle under different driving conditions. Testing was achieved using a **Škoda Kodiaq 2.0 TDI (110 kW, 2019)** communicating over a CAN bus at 500 kbit/s.

The entire measurement lasted about 13 minutes, comprising various driving phases:⁸

- Cold engine start and gentle pull-away from rest
- Smooth city driving with almost constant speed
- Sustained cruise at 50 km h⁻¹ to establish a steady-state baseline
- Full-throttle acceleration to 100 km h⁻¹ to capture peak engine loading
- Urban driving with mixed throttle and brake inputs
- Stationary high-RPM run with the vehicle at rest to capture peak engine speed without load

OBD-II data were sampled at a polling interval of 500 ms per PID and exported to a CSV file for post-processing. The acquired parameters, together with their OBD-II Service 01 PID identifiers and physical interpretation, are listed in Table 3.5.

Table 3.5: OBD-II parameters acquired during the test drive

PID	Parameter	Unit	Description
0x0C	Engine speed	rpm	Rises with throttle input and drops at each gearshift.
0x0D	Vehicle speed	km h ⁻¹	Wheel speed reported via the ABS control unit.
0x05	Coolant temperature	°C	Cylinder-head coolant temperature. Rises gradually from cold start and stabilizes once the thermostat opens.
0x0F	Intake air temp.	°C	Air temperature downstream of the intercooler.
0x46	Ambient temp.	°C	External air temperature as estimated by the ECU.
0x10	Mass air flow	g/s	Air mass entering the intake manifold per second. Peaks mirrors the engine load profile.
0x11	Throttle position	%	Normalized throttle opening, directly reflecting driver demand. Reaches maximum during the acceleration.
0x04	Engine load	%	ECU-calculated ratio of current to maximum available torque.
0x0B	Intake MAP	kPa	Absolute pressure in the intake manifold.

⁸Photographs taken from the vehicle cockpit during characteristic phases of the session are included in Appendix D as corroborating evidence for correct logged values.

The recorded data are presented in three figures. Figure 3.7 shows engine speed and vehicle speed as functions of time, with the session divided into clearly delimited driving phases for readability. During the initial city driving, the engine and car speeds are relatively low, showing oscillations resulting from multiple pedal presses. During the accelerating stage, there is a simultaneous rise in both plots, whereas during cruising, both become stable.

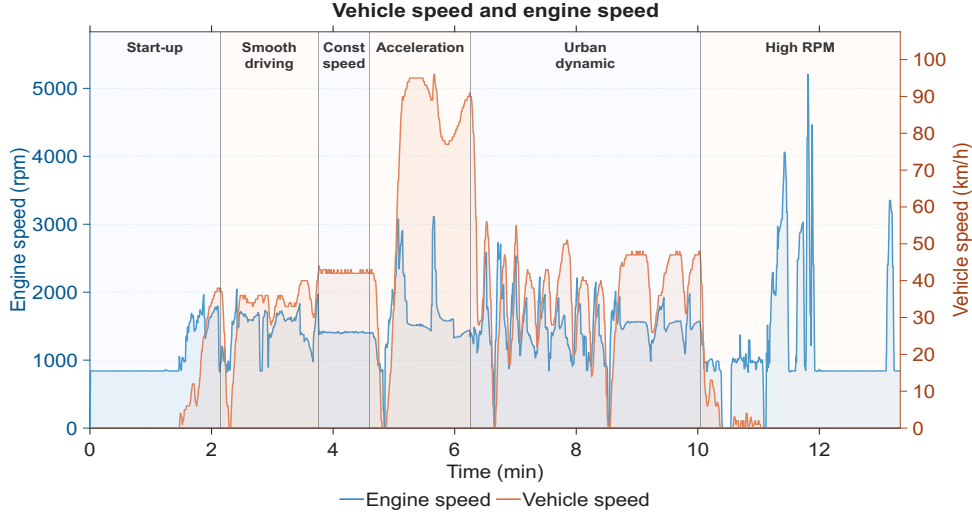


Figure 3.7: Recorded engine speed and vehicle speed over the test session.

Figure 3.8 presents throttle position, engine load, and intake manifold pressure over the same time base. During the full-throttle acceleration run, all three quantities rise sharply and in a physically consistent manner, with MAP climbing well above the ambient value of approximately 100 kPa, confirming active turbocharger operation. During steady-state cruise, engine load settles at a moderate level while throttle position remains nearly constant.

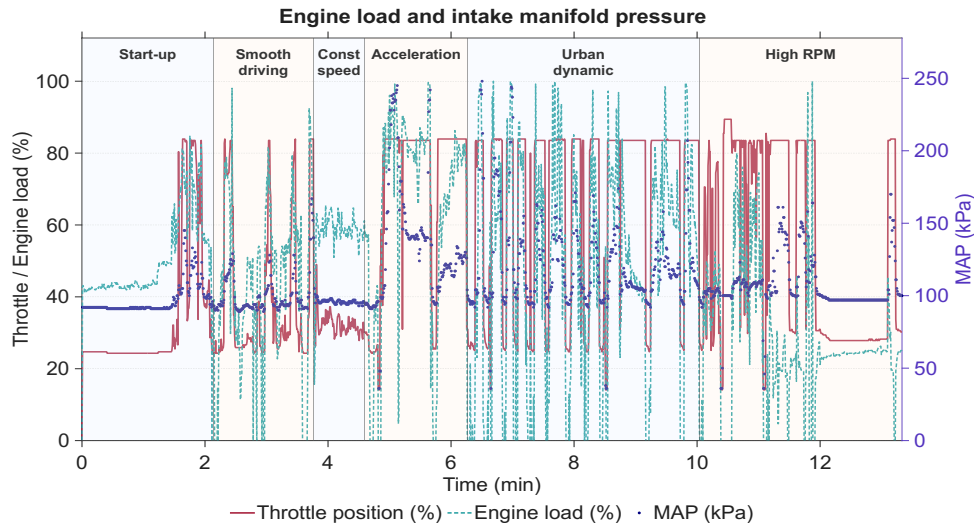


Figure 3.8: Throttle position, engine load, and intake manifold pressure.

Figure 3.9 covers coolant temperature, intake air temperature, ambient temperature, and mass air flow. Coolant temperature rises steadily from the cold-start value and stabilises once the thermostat opens, while the MAF trace closely mirrors the engine load profile throughout the session.

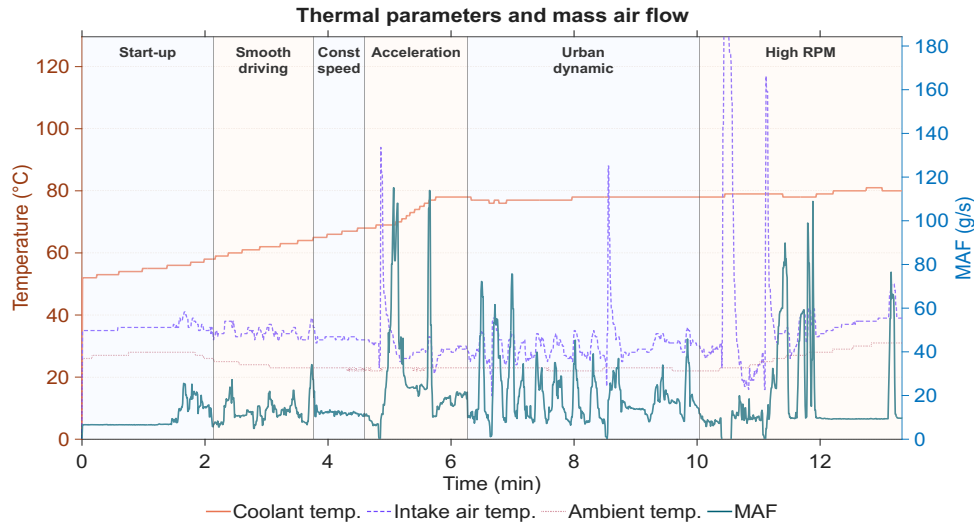


Figure 3.9: Coolant temperature, intake air temperature, ambient temperature, and mass air flow rate.

3.7.3 Power Measurements

In this chapter, the CAN Bus Sniffers's power consumption was evaluated to obtain real-world data for the power management. The measurements were performed using the Nordic Power Profiler Kit II (PPK2), which offers a high sampling rate of 100 kHz. Because this rapidly generates millions of data points, screenshots from the Nordic Power Profiler application are visualized. Exporting and externally downsampling such massive datasets risks losing crucial short-duration peaks, whereas the native app safely preserves them using an optimized min/max envelope approach.

Figure 3.10 illustrates the current draw during the LTE module's initialization, network registration, and the transmission of a TCP payload. During the initial boot and initialization phase, the current fluctuates between 60 mA and 115 mA. This is followed by a stabilization period with an idle current draw of approximately 60 to 80 mA. A significant peak of roughly 400 mA is then observed during the actual TCP packet transmission.

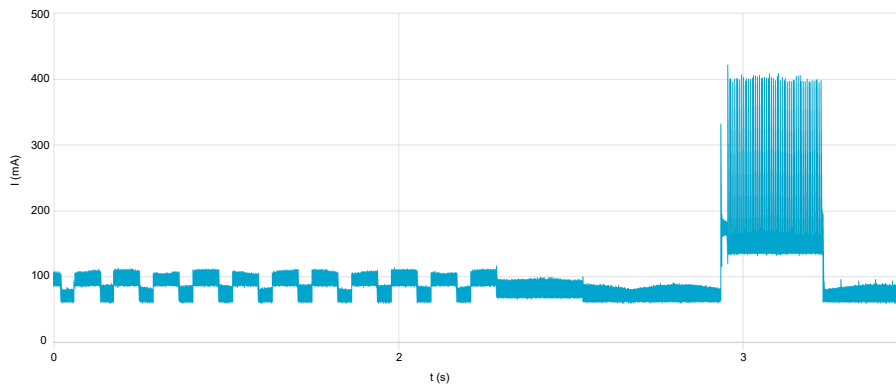


Figure 3.10: Current draw during LTE module initialization and TCP payload transmission.

The system's power consumption in different low-power states is compared in Figure 3.11. During the light sleep phase, the measured current draw settled at approximately 26 mA. The subsequent deep sleep period showed a marginally lower consumption of around 20 mA. The relatively modest difference between the two modes is likely attributable to residual peripheral activity—most notably the LTE modem has fully completed its shutdown before the sleep transition occurred.

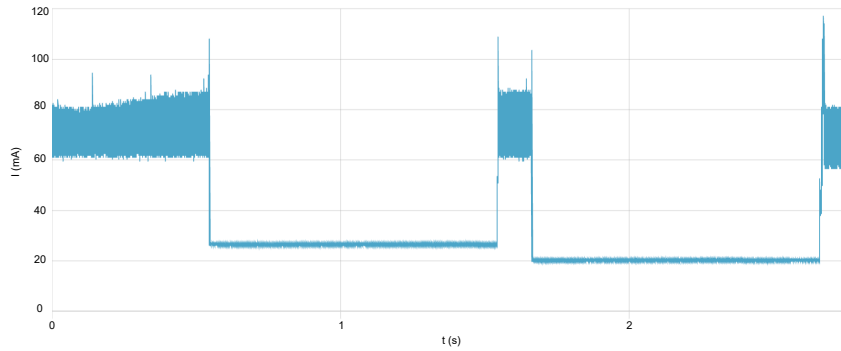
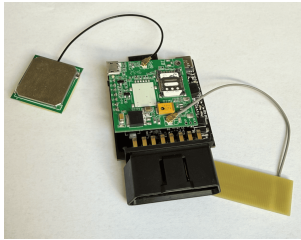


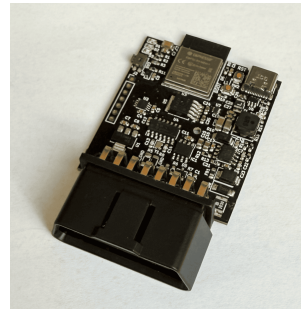
Figure 3.11: System power consumption comparison between light sleep and deep sleep modes.

3.8 Final Devices

As mentioned before, two prototypes were assembled: one equipped with an LTE module to handle direct connection to the server, and a second variant functioning solely as a Wi-Fi Access Point (AP) utilizing the ESP32 captive portal.



LTE device



Wi-Fi AP device

3.8.1 Web Server Endpoint

The data from the diagnostic device is transmitted and visualized on a custom web dashboard. The main endpoint receiving the telemetry data is available at:

<https://canbussnifferv1-production.up.railway.app>

3.8.2 Power Management

Since the device operates continuously in the automotive environment, power efficiency is critical. To prevent draining the vehicle's battery when the engine is turned off, firmware uses a three-state FSM driven by CAN traffic detection: DRIVING $\rightarrow$ IDLE_AWAKE after 60 s of bus silence $\rightarrow$ LIGHT_SLEEP after 5 min idle. In sleep, the modem enters AT+CSCLK=1 and the ESP32-S3 enters native light sleep with RAM retention; the device wakes every 60 s to recheck CAN Bus. Average current draw per phase is summarized in Table 3.6.

Table 3.6: Measured current draw per operating phase.

FSM Phase	Measured current
DRIVING	≈ 400 mA
IDLE_AWAKE	≈ 86 mA
LIGHT_SLEEP	≈ 26 mA
DEEP_SLEEP	≈ 20 mA

For calculation, a typical 60 Ah vehicle battery was used. The maximum safe parking endurance T is derived from the usable battery capacity C_{usable} , limiting depth-of-discharge to 50 % to retain cranking ability:

$$T = \frac{C_{\text{usable}}}{I_{\text{sleep}}} = \frac{0.5 \times 60,000 \text{ mAh}}{26 \text{ mA}} \approx 1,154 \text{ h} \approx 48 \text{ days}. \quad (3.2)$$

This calculated time is well above the typical use-case. This proves that light sleep mode is enough to protect the car's battery during long-time parking.

3.8.3 Data Payload

Each transmission consists of a fixed overhead O and a variable payload $P(n)$ depending on the number of supported PIDs n :

$$B_{\text{send}} = O + P(n). \quad (3.3)$$

The fixed overhead covers the TLS 1.2 handshake, HTTP POST header, and server response:

$$O = O_{\text{TLS}} + O_{\text{HTTP}} + O_{\text{resp}} \approx 5500 + 225 + 150 = 5875 \text{ B}. \quad (3.4)$$

The payload size differs between the two packet types. The fast packet encodes only four parameters; the full packet encodes all n supported PIDs:

$$P_{\text{fast}} = B_{\text{base}} + 4\bar{k}, \quad (3.5)$$

$$P_{\text{full}}(n) = B_{\text{base}} + n\bar{k}, \quad (3.6)$$

where $B_{\text{base}} \approx 30 \text{ B}$ is the fixed JSON envelope (**nonce**, **type**, **lat**, **lon**) and $\bar{k} \approx 16 \text{ B}$ is the mean length of one key-value pair. For the Škoda Kodiaq

tested in this work, 9 PIDs were identified as supported. Substituting $n = 9$ into equations (3.5) and (3.6):

$$\begin{aligned} P_{\text{fast}} &= 30 + 4 \times 16 = 94 \text{ B}, \\ P_{\text{full}}(9) &= 30 + 9 \times 16 = 174 \text{ B}. \end{aligned}$$

Field measurements confirmed values of 95 B and 179 B respectively. The system sends two fast packets and one full packet every 6 s, giving:

$$V_{\text{hour}} = \frac{3600}{6} (2 B_{\text{fast}} + B_{\text{full}}) = 600 \cdot (2 \times 5969 + 6049) \approx 10.3 \text{ MB/h.} \quad (3.7)$$

The payload contribution alone (excluding TLS overhead) is $\approx 0.2 \text{ MB/h}$. The dominant term is the TLS handshake re-established on every POST, a consequence of the `Connection: close` behavior of the A7670E HTTPINIT stack. Table 3.7 summarizes the projected consumption for 4 hours of daily driving.

Table 3.7: Projected cellular data consumption (4 h/day, $n = 9$ PIDs).

Period	Payload only	Total (incl. TLS)
Day (4 h)	0.7 MB	41.2 MB
Month	20 MB	1.24 GB
Year	0.24 GB	14.7 GB



Chapter 4

Conclusion

This bachelor thesis presented a comprehensive theoretical study of the CAN bus interface and its protocols, along with their large-scale implementation in the automotive industry. Additionally, serial communication buses used in automotive devices such as FlexRay, LIN, Automotive Ethernet, and MOST were presented. The theoretical section also provided a detailed explanation of the OBD-II standard and the process of decoding diagnostic messages.

The practical part of the thesis focused on the design and development of a custom diagnostic and sniffing device utilizing a microcontroller, a CAN FD controller, and an external LTE/GPS module. This phase included custom firmware development for acquiring telemetry data directly from the CAN bus, configuring a backend server for data logging, and creating a web user interface for data visualization.

Testing and validation were also part of the thesis, encompassing the discussion of security measures, the evaluation of power management strategies through current consumption measurements, and the validation of the acquired telemetry data.

The final outcome of this thesis consists of two prototype devices. The primary device uses an LTE cellular modem to connect to the backend server that allows wireless data transmission and visualization on a web dashboard. The second variant serves as a stand-alone offline device that uses the microcontroller as a local Access Point with its own web server having a captive portal.

The thesis also outlines potential future extensions of the system, such as the implementation of a bidirectional diagnostic interface capable of flashing vehicle Electronic Control Units or precise positioning using Cell ID service.

All objectives of the thesis were achieved. Despite the inherent limitations of a prototype stage, the developed devices establish a robust and functional foundation that allows for future hardware implementations and further software optimization to ensure long-term stability.



Bibliography

- [1] K AMARDEEP and DPS KUMAR. *Most Media Oriented Systems Transport: A Comprehensive Analysis of the Automotive Multimedia Networking Protocol*. 7. Accessed: 2025-01-25. 2021, pp. 1830–1835. Available from: <https://url-shortener.me/A71F>.
- [2] CADENCE DESIGN SYSTEMS. *Can bus history at a glance*. 2022. Available from: <https://resources.pcb.cadence.com/blog/2022-can-bus-history-at-a-glance> (visited on 01/25/2025).
- [3] CAN IN AUTOMATION (CIA). *History of CAN technology*. 2026. Available from: <https://www.can-cia.org/can-knowledge/history-of-can-technology> (visited on 01/24/2026).
- [4] CHIEF DELPHI. *CAN Bus Topology Diagram*. [Accessed: 2026-05-02]. 2023. Available from: <https://www.chiefdelphi.com/t/can-bus-topology/423564/2>.
- [5] CIRCUITBREAD. *Understanding CAN: A Beginner's Guide to the Controller Area Network Protocol*. [Accessed: 2026-05-02]. 2020. Available from: <https://www.circuitbread.com/tutorials/understanding-can-a-beginners-guide-to-the-controller-area-network-protocol>.
- [6] CSS ELECTRONICS. *CAN Bus Explained - A Simple Intro [2024]*. Accessed: 2025-01-25. CSS Electronics. 2024. Available from: <https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial>.
- [7] CSS ELECTRONICS. *CAN Bus: The Ultimate Guide*. 2024. Available from: <https://www.csselectronics.com/pages/can-bus-ultimate-guide> (visited on 01/25/2025).
- [8] CSS ELECTRONICS. *OBD2 Explained - A Simple Intro [2024]*. Technical Guide. Accessed: 2025-01-31. 2024. Available from: <https://www.csselectronics.com/pages/obd2-explained-simple-intro>.
- [9] CSS ELECTRONICS. *OBD2 Explained — A Simple Intro*. [Accessed: 2026-05-02]. 2024. Available from: <https://www.csselectronics.com/pages/obd2-explained-simple-intro>.

- [10] RI DAVIS, A BURNS, RJ BRIL, and JJ LUKKIEN. “Controller Area Network (CAN) schedulability analysis: Refuted, revisited and revised”. In: *Real-Time Systems* 35.3 (2007), pp. 239–272.
- [11] DRÁTEK.CZ. *Bluetooth automobilová diagnostická jednotka ELM327 pro OBD2*. Czech. Accessed: 2026-05-02. 2026. Available from: <https://dratek.cz/arduino-platforma/5004-bluetooth-automobilova-diagnosticka-jednotka-elm327-pro-obd2.html>.
- [12] GRID CONNECT. *CAN FD: The Next Big, Fast Thing*. Blog post. Accessed: 2025-01-25. 2024. Available from: <https://www.gridconnect.com/blogs/news/can-fd-the-next-big-fast-thing>.
- [13] J HOPKINS. *What is an Electronic Control Unit in Automotive Systems?* 2024. Available from: <https://www.totalphase.com/blog/2024/05/what-is-electronic-control-unit-automotive-systems/> (visited on 01/26/2026).
- [14] ISO 11898-1:2015 *Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signaling*. Geneva, Switzerland: International Organization for Standardization, 2015.
- [15] ITBIZNES. *T-Mobile Smart Car — lokalizacja i monitoring pojazdów*. Polish. [Accessed: 2026-05-02]. 2018. Available from: <https://itbiznes.pl/sprzet/t-mobile-smart-car-lokalizacja-monitoring-pojazdow/>.
- [16] M JEDH, L BEN OTHMANE, N AHMED, and B BHARGAVA. “Detection of Message Injection Attacks Onto the CAN Bus Using Similarities of Successive Messages-Sequence Graphs”. In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 4133–4146. DOI: [10.1109/TIFS.2021.3098162](https://doi.org/10.1109/TIFS.2021.3098162).
- [17] KAMERAK.CZ. *ELM327 Bluetooth v2.1 OBDII autodiagnostika*. Czech. [Accessed: 2026-05-02]. 2026. Available from: <https://www.kamerak.cz/detail-486-1-D8015011/elm327-bluetooth-v2-1-obdii-car-diagnostic.html>.
- [18] KVASER AB. *CAN Protocol Tutorial*. Accessed: 2025-01-25. 2024. Available from: <https://kvaser.com/can-protocol-tutorial/>.
- [19] L LIU, W LIU, and K PENG. “Vehicle Information Collection and Communication System Based on CAN Bus and GPS”. In: *Journal of Physics: Conference Series* 2023.1 (2021), p. 012045. DOI: [10.1088/1742-6596/2023/1/012045](https://doi.org/10.1088/1742-6596/2023/1/012045).
- [20] MICROST. *The CAN Bus — Physical Layer*. [Accessed: 2026-05-02]. 2026. Available from: https://www.microst.it/Tutorial/can_pl.htm.
- [21] NERD CORNER. *Cyclic Redundancy Check (CRC) in C Code*. [Accessed: 2026-05-02]. 2023. Available from: <https://nerd-corner.com/cyclic-redundancy-check-in-c-code/>.

- [22] OBDELEVEN CZ. *OBDeleven — Oficiální e-shop pro ČR a SR*. Czech. [Accessed: 2026-05-02]. 2026. Available from: <https://www.obd-eleven.cz/>.
- [23] P PIMPLE. “Sniffing the Automotive CAN Bus for Real-time Data-logging and Real Time Diagnostics Display”. In: *2018 International Conference on Smart Electric Drives and Power System (ICSEDPS)*. IEEE, 2018, pp. 167–170. DOI: [10.1109/ICSEDPS.2018.8536068](https://doi.org/10.1109/ICSEDPS.2018.8536068).
- [24] ROBERT BOSCH GMBH. *CAN Specification, Version 2.0*. Available online via University of California, Riverside. Accessed: 2026-05-17. 1991. Available from: <http://esd.cs.ucr.edu/webres/can20.pdf>.
- [25] ROBERT BOSCH GMBH. *Engine Control Unit (ECU) Solutions*. [Accessed: 2026-05-02]. 2026. Available from: <https://www.bosch-mobility.com/en/solutions/control-units/engine-control-unit/>.
- [26] ROHDE & SCHWARZ. *Low-speed Automotive Bus Testing*. [Accessed: 2026-05-02]. 2026. Available from: https://www.rohde-schwarz.com/us/solutions/automotive-testing/in-vehicle-networks-and-ecu-testing/low-speed-automotive-bus-testing/low-speed-automotive-bus-testing_231836.html.
- [27] T SABER, S VENTICINQUE, A AMATO, and L ROCCO. *Generic model of a car CAN bus — Multiple subnetworks with different functionality*. Accessed: 2026-05-02. 2017. Available from: https://www.researchgate.net/figure/Generic-model-of-a-car-CAN-bus-Multiple-subnetworks-with-different-functionality-are_fig1_320837400.
- [28] M SIERGIEJCZYK, A ROSIŃSKI, and M WAWRZYŃSKI. “Analysis of the vehicle operation process including diagnostic information”. In: *MATEC Web of Conferences* 182 (2018), p. 01028. DOI: [10.1051/mateconf/201818201028](https://doi.org/10.1051/mateconf/201818201028).
- [29] SIGNAL PROFI. *Intelligentní 4G GPS lokátor do OBD s doživotně bezplatným provozem SIZZAPP*. Czech. Accessed: 2026-05-02. 2026. Available from: <https://www.signalprofi.cz/intelligentni-4g-gps-lokator-do-obd-s-dozivotne-bezplatnym-provozem-sizzapp>.
- [30] SIGNAL PROFI. *Intelligentní 4G GPS lokátor do OBD s doživotně bezplatným provozem SIZZAPP*. Czech. [Accessed: 2026-05-02]. 2026. Available from: <https://www.signalprofi.cz/intelligentni-4g-gps-lokator-do-obd-s-dozivotne-bezplatnym-provozem-sizzapp>.
- [31] SIMMA SOFTWARE. *ISO 15765-4 Code: The Complete Guide to OBD-II over CAN*. Technical Guide. Accessed: 2025-01-31. 2024. Available from: <https://simmasoftware.com/iso-15765-4-code-the-complete-guide-to-obd-II-over-can/>.
- [32] N SINGARAJU. “Real-Time Fleet Tracking and Diagnostic System Using CAN Bus and GPS”. In: *International Journal on Science and Technology (IJSAT)* 16.3 (2025). DOI: [10.71097/ijSAT.v16.i3.6895](https://doi.org/10.71097/ijSAT.v16.i3.6895).

- [33] T-MOBILE CZECH REPUBLIC. *Chytré auto (Smart Car)*. Czech. Accessed: 2026-05-02. 2026. Available from: <https://www.t-mobile.cz/chytre-auto>.
- [34] TAOGLAS. *FXUB62 — 4G LTE Flexible PCB Antenna*. [Accessed: 2026-05-02]. 2026. Available from: <https://www.taoglas.com/product/fxub62-4g-lte-flex-pcb-antenna/>.
- [35] TEXAS INSTRUMENTS. *Introduction to the CAN Bus*. SLOA101B. Application Report. 2016. Available from: <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>.
- [36] TINYTRONICS. *GPS Antenna with u.FL Connector*. [Accessed: 2026-05-02]. 2026. Available from: <https://www.tinytronics.nl/en/communication-and-signals/wireless/antennas/gps/gps-antenne-with-ufl-connector>.
- [37] VERIZON CONNECT. *FAQ: Where can I find the OBD-II port?* [Accessed: 2026-05-02]. 2024. Available from: <https://install.verizonconnect.com/hc/en-us/articles/360060462133-FAQ-Where-can-I-find-the-OBd-II-port>.
- [38] VOLTAS IT. *OBDeleven: Next-generation diagnostic tool*. Accessed: 2026-05-02. 2026. Available from: <https://obdeleven.com/>.
- [39] WIKIMEDIA COMMONS. *CAN bus frame with stuff bit and correct CRC*. [Accessed: 2026-05-02]. 2026. Available from: https://en.wikipedia.org/wiki/CAN_bus#/media/File:CAN-bus-frame-with-stuff-bit-and-correct-CRC.png.
- [40] S WOO, HJ JO, and DH LEE. “A Practical Wireless Attack on the Connected Car and Security Protocol for In-Vehicle CAN”. In: *IEEE Transactions on Intelligent Transportation Systems* (2014), pp. 1–14. DOI: [10.1109/TITS.2014.2351612](https://doi.org/10.1109/TITS.2014.2351612).
- [41] H ZELTWANGER. “Standards and Specifications: CAN XL and beyond”. In: *CAN Newsletter Online* (Sept. 2024). CiA Publication. Accessed: 2025-01-25, pp. 4–6. Available from: https://www.can-cia.org/fileadmin/cia/documents/publications/cnlm/september_2024/cnlm_24-3_p4_standards_and_specifications_holger_zeltwanger_cia.pdf.

Appendix A

Component Datasheets

This appendix provides links to the official datasheets of the key components used in the hardware design of the diagnostic device.

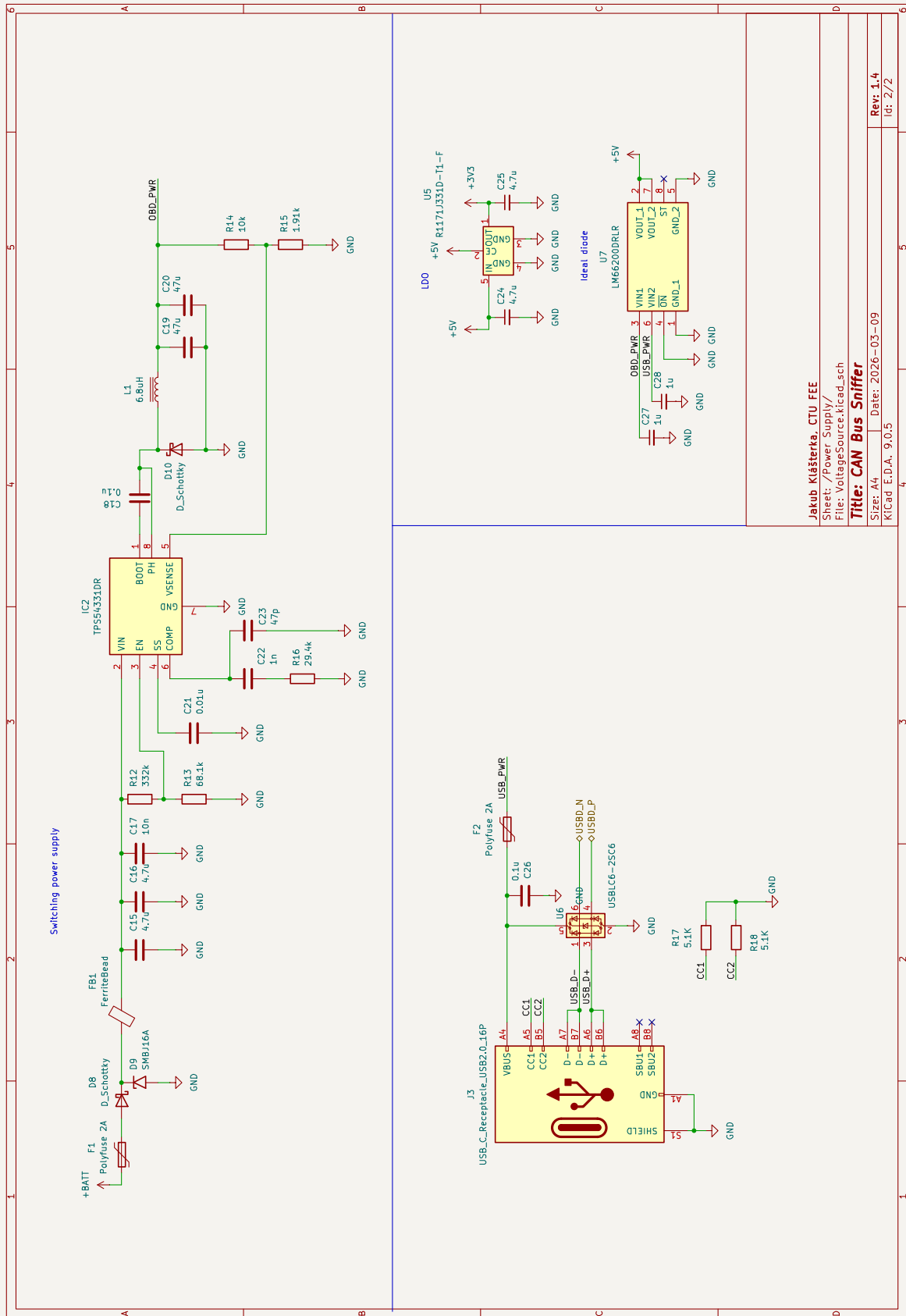
- D.1 ESP32-S3-MINI-1-N8** (Espressif Systems) — Main microcontroller module.
- D.2 MCP2518FD** (Microchip Technology) — External CAN FD controller with SPI interface.
- D.3 A7670E** (SIMCom Wireless Solutions) — LTE Cat-1 cellular module with integrated GNSS.
- D.4 TPS54331** (Texas Instruments) — Step-down DC/DC converter for the power supply subsystem.
- D.5 ATA6563** (Microchip Technology) — CAN FD transceiver.
- D.6 R1171J** (Ricoh/Nisshinbo) — Low-dropout (LDO) linear regulator.
- D.7 LM66200** (Texas Instruments) — Ideal diode controller for reverse polarity protection.



Appendix B

Complete Hardware Schematic

In this appendix, the complete electrical schematic of the developed diagnostic device is presented. The schematic was designed in accordance with the automotive requirements discussed in Chapter 3.





Appendix C

Dashboard Screens

This appendix provides screenshots of all dashboard screens and the advanced diagnostics interface described in Section [3.6.1](#).

Cockpit Screen

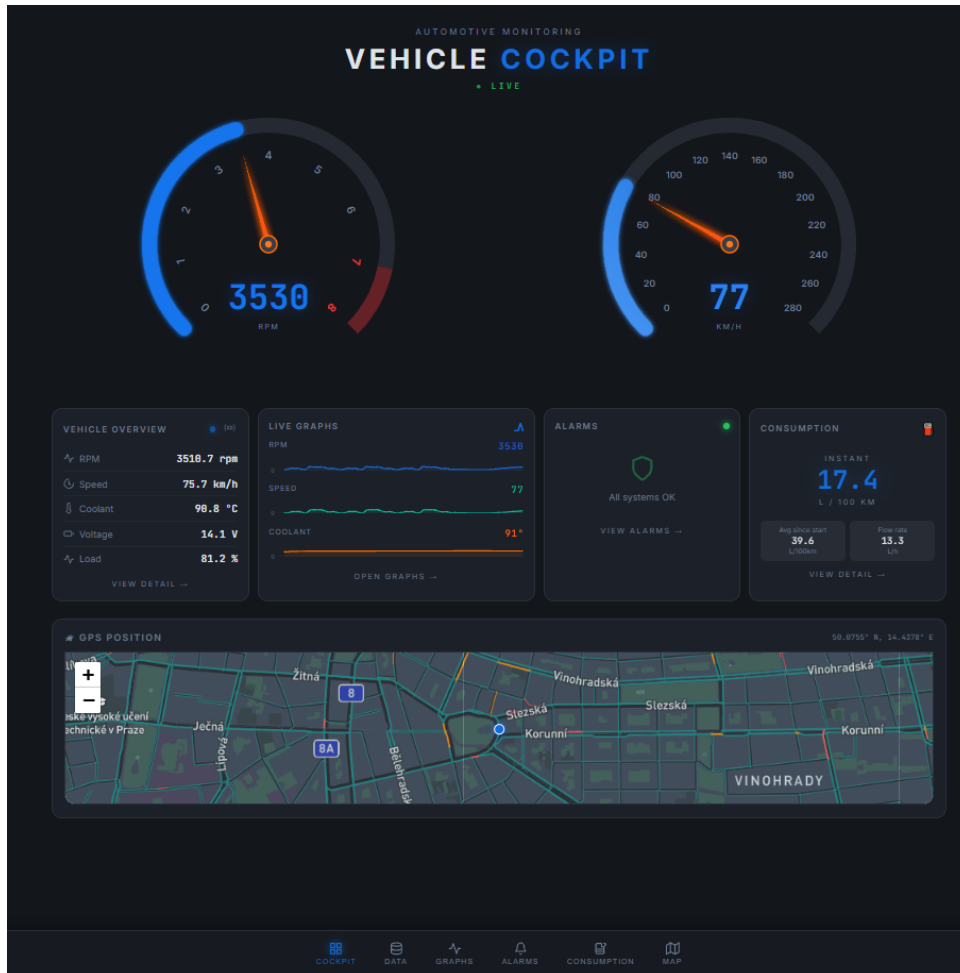


Figure C.1: Cockpit screen — live radial gauges and summary widgets

Vehicle Overview Screen

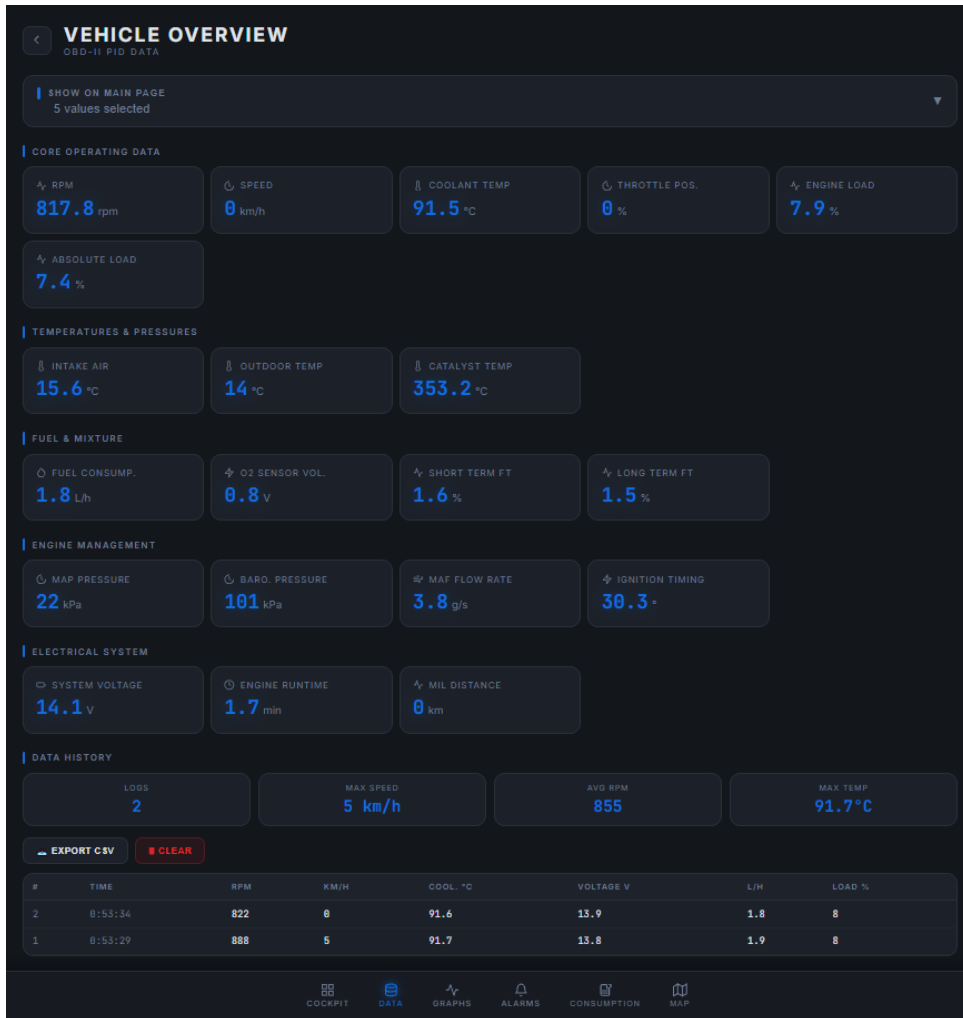


Figure C.2: Vehicle Overview screen — PID values and CSV export

Live Graphs Screen

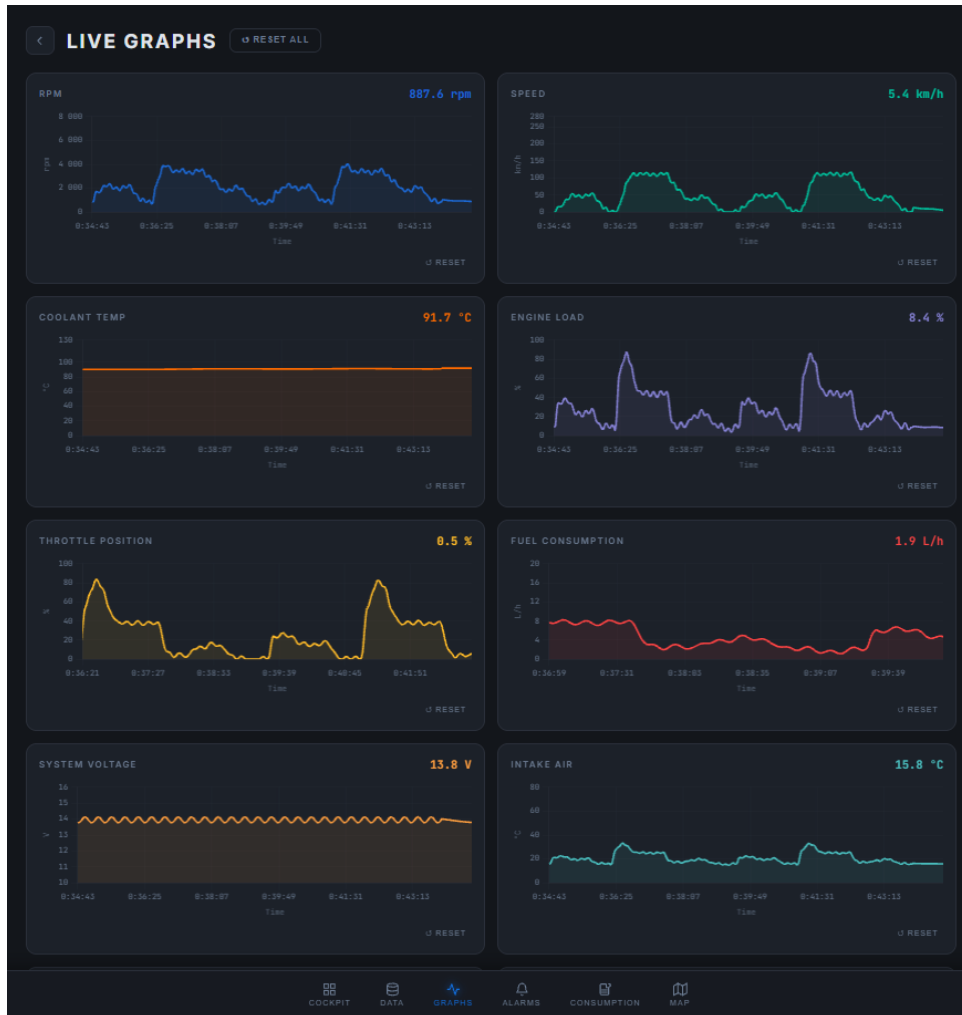


Figure C.3: Live Graphs screen — time series plots with zoom support

Fuel Consumption Screen



Figure C.4: Fuel Consumption screen — instantaneous and average counters

GPS Map Screen

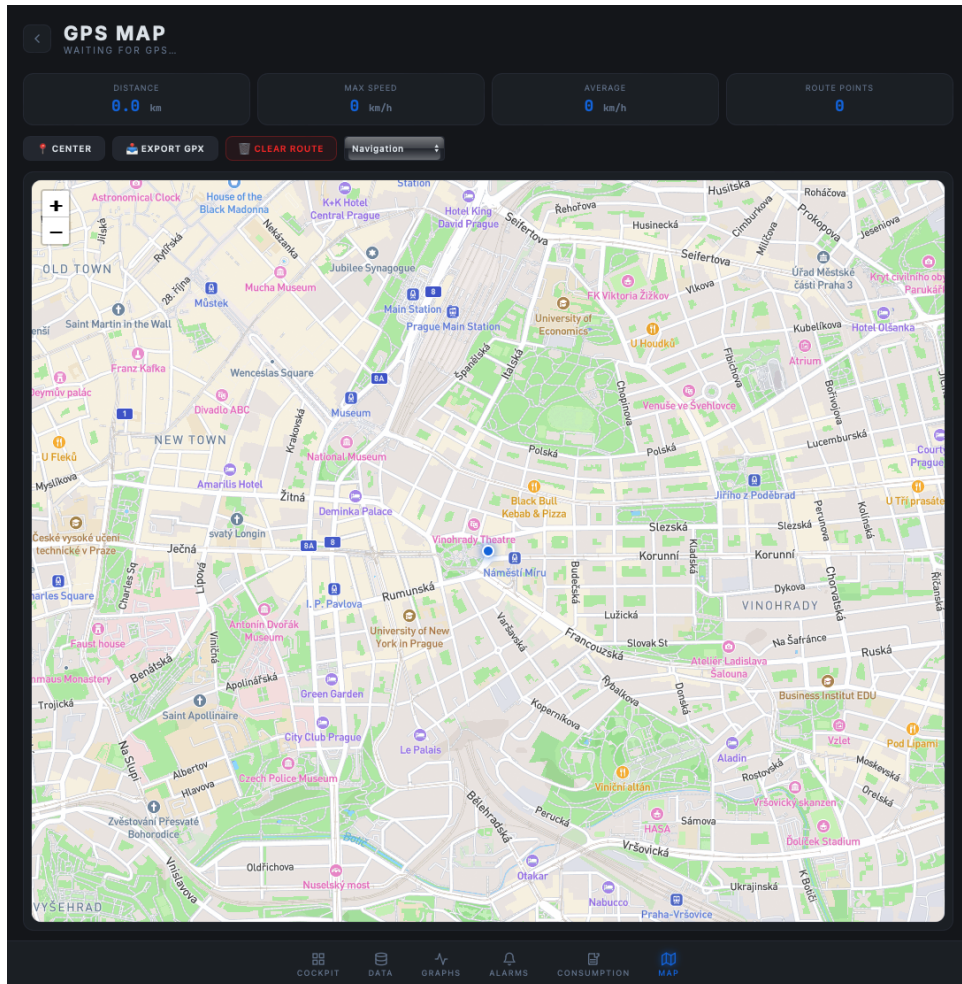


Figure C.5: GPS Map screen — real-time vehicle position and route export

Alarms Screen

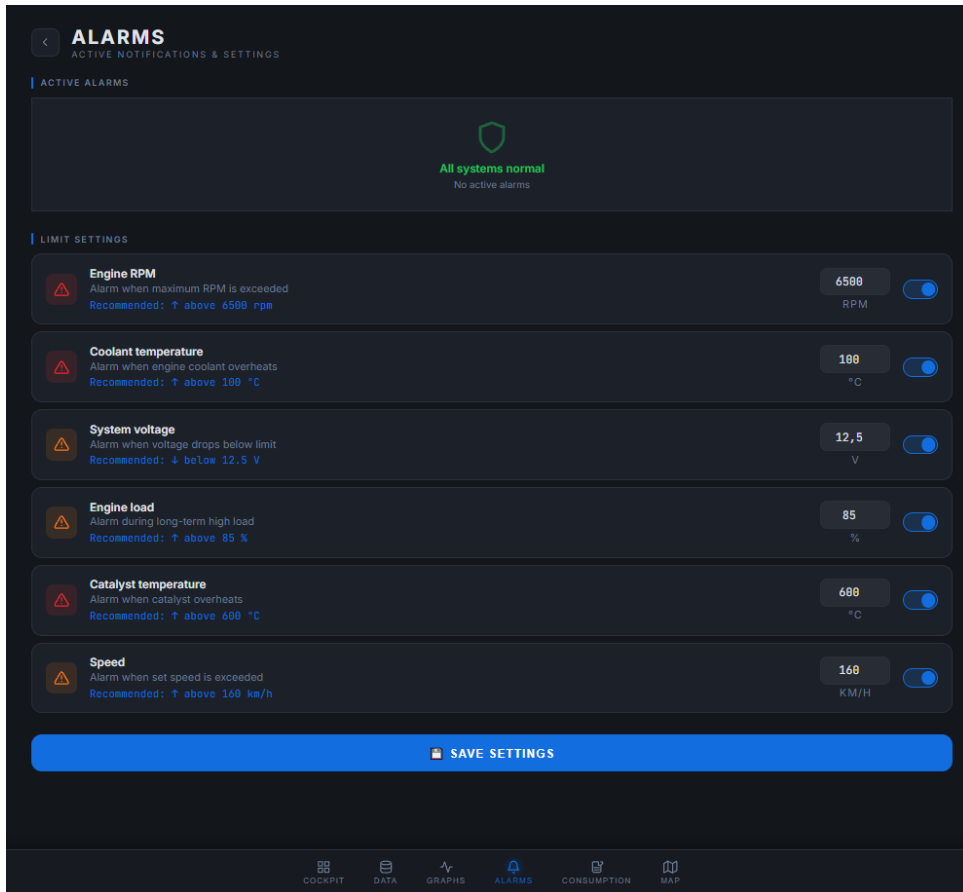


Figure C.6: Alarms screen — configurable threshold limits

Advanced Diagnostics Interface

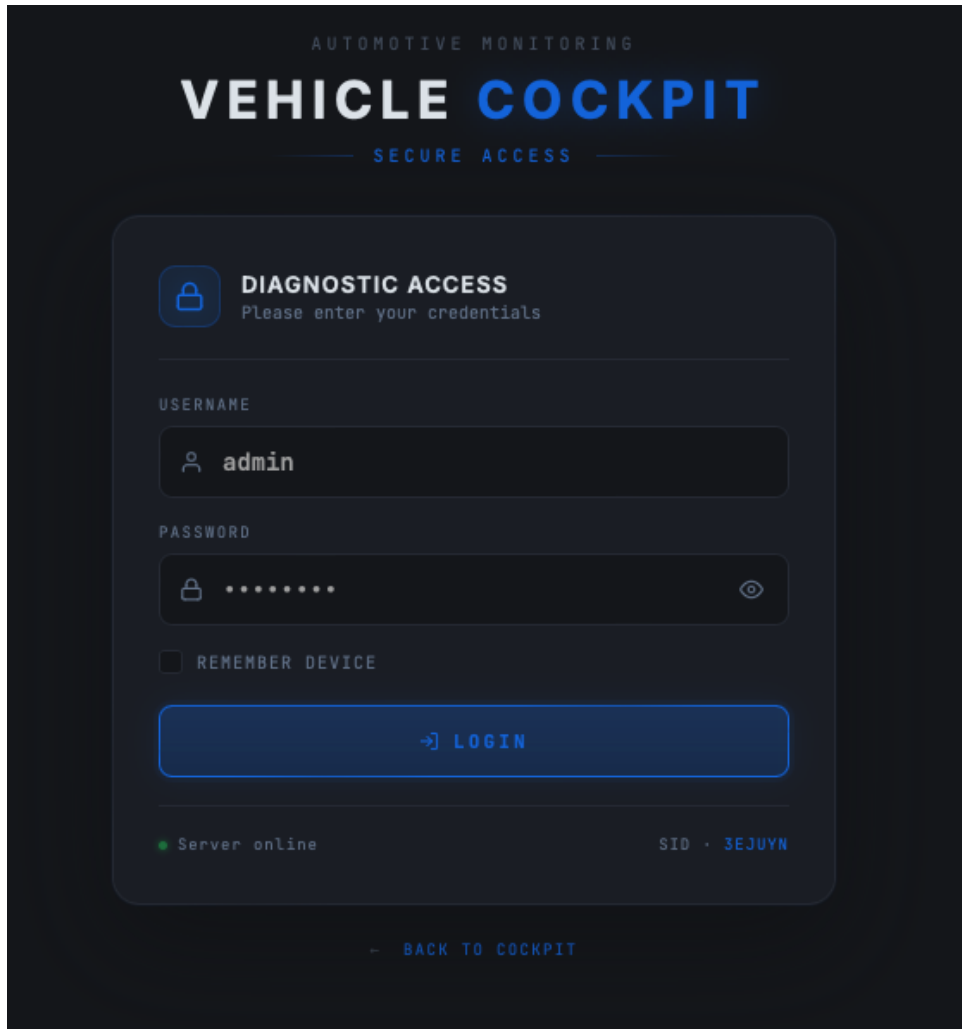


Figure C.7: Login screen — access restriction for the diagnostics interface

DIAGNOSTICS

CAN · UDS · CODING

CAN BAUD RATE:

1250 KBIT/S

LOG OUT

FRAME BUILDER

LIVE MONITOR

UDS / CODING

LIBRARY

SERIAL

FRAME CONSTRUCTION

CAN 2.0 · STD

CAN ID (HEX)

7DF

DLC (LENGTH)

8

byte count

FORMAT

Standard (11b)

ID Length

11 bit

RTR (REMOTE)

CAN FD

BRS (BITRATE SWITCH)

DATA PAYLOAD (HEX)

0000000000000000

RAW FRAME

7DF # [8] · 00 00 00 00 00 00 00 00

SEND

LOOP

SAVE

CLEAR

BIT MATRIX

64 BIT

B0	0	0	0	0	0	0	0	0	0x00
B1	0	0	0	0	0	0	0	0	0x00
B2	0	0	0	0	0	0	0	0	0x00
B3	0	0	0	0	0	0	0	0	0x00
B4	0	0	0	0	0	0	0	0	0x00
B5	0	0	0	0	0	0	0	0	0x00
B6	0	0	0	0	0	0	0	0	0x00
B7	0	0	0	0	0	0	0	0	0x00

SIGNAL EXTRACTION

START BIT

0

LENGTH (BITS)

8

BYTE ORDER

Intel (LE)

TYPE

Unsigned

FACTOR

1

OFFSET

0

UNIT

e.g. °C

PHYSICAL VALUE

0

RAW

0

0x0

Figure C.8: Diagnostic interface

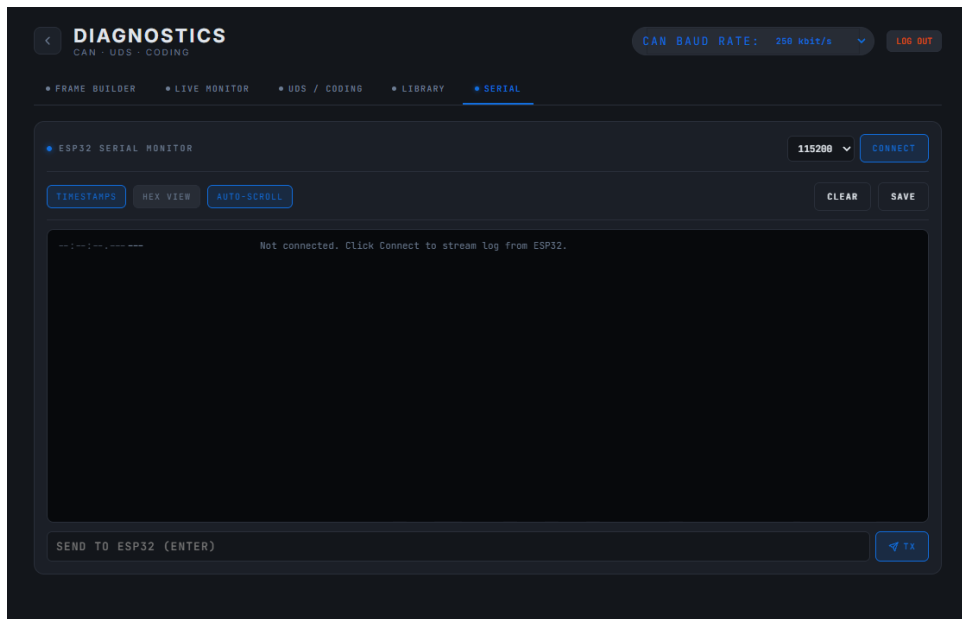


Figure C.9: Serial Monitor

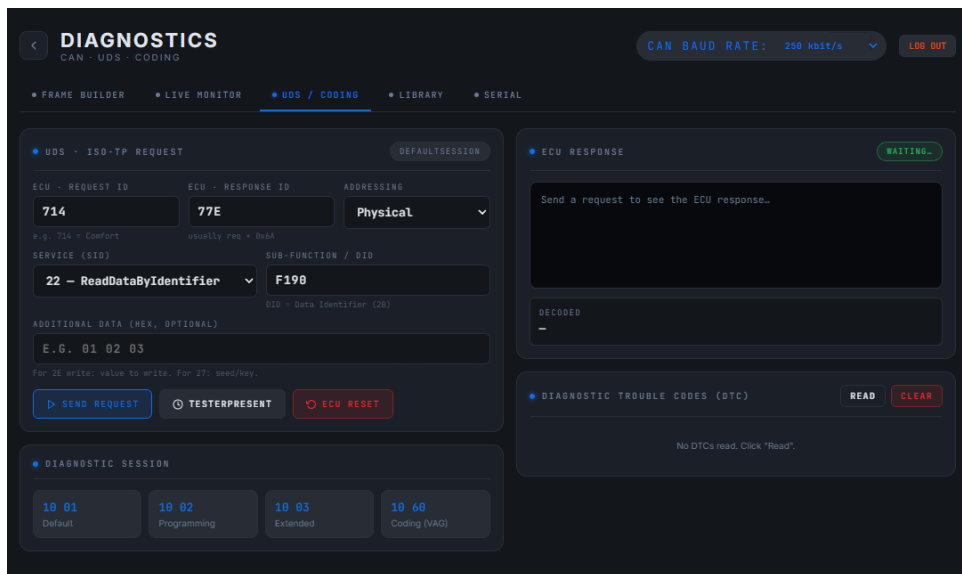


Figure C.10: UDS & Coding — ISO-TP request builder

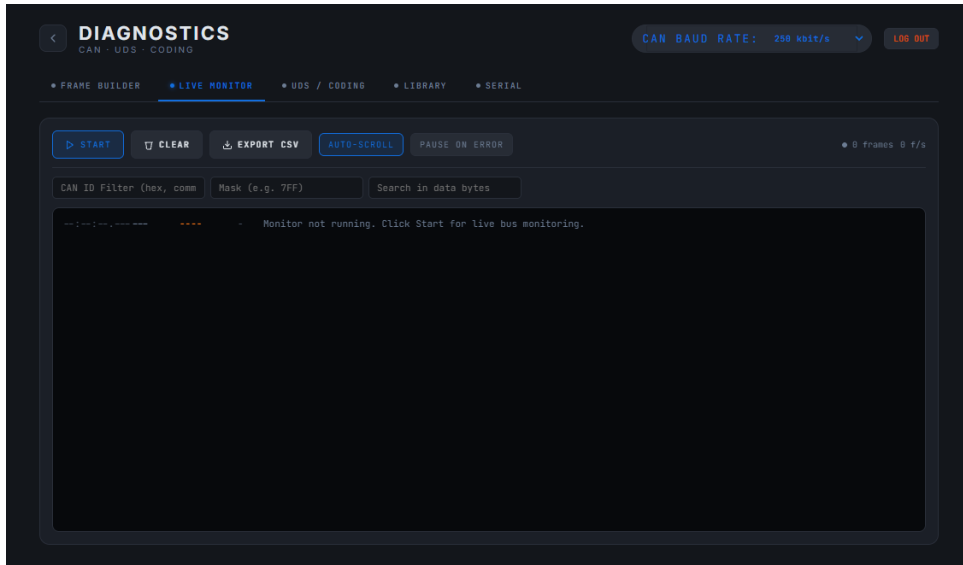


Figure C.11: Live monitor — real-time CAN traffic display

Appendix D

Test Drive Cockpit Photographs

This appendix provides photographs taken from the vehicle cockpit during characteristic phases of the measurement session described in Section 3.7.2. The displayed instrument cluster values serve as corroborating evidence that the OBD-II logged data reflect the actual vehicle readings.

45 km h⁻¹ Cruise



Figure D.1: Instrument cluster during 45 km h⁻¹ cruise

Full-Throttle Acceleration to 100 km h^{-1}



Figure D.2: Instrument cluster during full-throttle acceleration to 100 km h^{-1}

50 km h^{-1} Steady-State



Figure D.3: Instrument cluster during 50 km h^{-1} steady-state cruise

High-RPM Stationary Run



Figure D.4: Instrument cluster during stationary high-RPM engine run

Appendix E

Repository Structure

The complete project is stored in a public GitHub repository, organized into the following directories. The repository is available at:

https://github.com/Kubakl11/CAN_Bus_Sniffer_BP

Table E.1: Repository Structure

Directory	Contents
<code>project-root/</code>	Main repository root
<code>PCB-Design/</code>	PCB design files (schematics, layout)
<code>firmware/ LTE-version/ WiFi-AP-version/</code>	Embedded software for microcontroller Firmware version with LTE connectivity Firmware version running as WiFi Access Point
<code>server/ dashboard/</code>	Backend and data processing Web dashboard (HTML, CSS, JavaScript)
<code>3D-print-case/</code>	STL files of the printed case
<code>thesis/</code>	Thesis in PDF version

Appendix F

Usage of Artificial Intelligence tools

This appendix describes the specific Artificial Intelligence tools and models used during the development of this thesis:

- **Google Gemini (Gemini 1.5 Pro, 1.5 Flash, and 2.0 Flash/Pro):** Used for correcting text and grammar, refining vocabulary to achieve an academic tone, and formatting $\text{\LaTeX}$ syntax.
- **Claude Sonnet 4.6 and 4.7 Opus):** Used as a consultant for firmware development and server infrastructure setup. These models assisted in debugging and structuring code blocks.
- **Google NotebookLM and Scholar Labs:** Used during the research phase to identify relevant academic sources, summarize technical documentation, and find useful information for the theoretical background.

While these tools provided assistance, the overall system architecture, research methodology, and final implementation were carried out and verified by the author.